

The Geometry of Motion

Volume III: Biomechanics

From Rigid Bodies to Biological Systems



Preface

Draft Notice. This volume is under active development. Chapters 1–3 contain substantial content; Chapters 4–10 are structural outlines with preliminary material that will be expanded in future editions. Chapter 1 includes worked examples and verification exercises. Their numerical consistency does not establish empirical validation of the volume’s biological models.

The preceding volumes develop geometric and control-theoretic tools using declared mechanical abstractions. This volume asks which abstractions support questions about biological movement. Rigid segments and idealized joints can remain useful; muscle states, tendon compliance, contact, or tissue deformation must be added when the intended output requires them.

Compliance enters constitutive forces and can require additional coordinates; it is not itself an inertial contribution to the mass matrix. Selecting neural excitation as the input also changes the state equations and the meaning of zero-input drift. Chapter 1 establishes these distinctions and shows why a musculoskeletal model is not automatically smooth and control-affine in excitation.

The aim is mathematics constrained by physical evidence. Validation remains specific to the model, population, task, and quantity being predicted. A match to observed motion does not validate every inferred muscle force or tissue load.

Contents

Preface	i
Nomenclature	v
1 How Biology Differs From Engineering	1
1.1 The Rigid-Body Assumption and Its Limits	1
1.2 Compliance, Timescales, and the Chosen Output	2
1.2.1 Adding Deformation Requires Adding Dynamics	2
1.2.2 A Frequency Comparison With a Checkable Meaning	2
1.3 From Excitation to Joint Power	3
1.3.1 Force Definitions and Units	3
1.3.2 Moment Arms Must Preserve Power	4
1.4 Two Different Kinds of Redundancy	4
1.5 Choosing the Model by the Question	5
1.5.1 Rigid Segments With Generalized-Load Actuators	5
1.5.2 Rigid Segments With Muscle and Tendon States	5
1.5.3 Deformation or Tissue Stress as an Output	6
1.6 Mass Properties Need Accounting and Frames	6
1.6.1 A Complete Segment Inventory	6
1.6.2 Centre of Mass and Spatial Inertia	6
1.7 When the Control-Affine Framework Applies	7
1.7.1 Choose the Input Before Splitting Drift and Control	7
1.8 Connecting the Parts of the Swing Argument	8
2 Musculoskeletal Modeling Conventions	10
2.1 Anatomical Reference Frames	10
2.1.1 The Anatomical Position	10
2.1.2 Segment Reference Frames	11
2.1.3 Connection to Screw Theory	11
2.2 Joint Coordinate Systems	11
2.2.1 The Grood–Suntay Convention for the Knee	12
2.2.2 Multi-Representation Joint Angles	13
2.3 Body Segment Parameters	14
2.3.1 Regression Equations	14
2.3.2 Computing the Mass Matrix	14
3 Muscle Models	16
3.1 The Hill-Type Muscle Model	16
3.1.1 Model Architecture	16
3.1.2 The Force-Length Relationship	17
3.1.3 The Force-Velocity Relationship	20
3.1.4 Activation Dynamics	21
3.2 Tendon Mechanics	21

3.3	Musculotendon Equilibrium	21
3.4	The Complete Hill Model as a Dynamical System	22
4	Joint Kinematics and Soft Tissue	24
4.1	Beyond Ideal Joints	24
4.1.1	The Instantaneous Screw Axis	24
4.1.2	The Knee: Rolling, Sliding, and Coupled Rotation	25
4.1.3	The Shoulder: The Most Complex Joint	26
4.2	Ligament Models	27
4.3	Spinal Mechanics	27
5	Multi-Body Dynamics of Biological Chains	29
5.1	Open-Chain vs. Closed-Chain Biomechanics	29
5.1.1	Equations of Motion for Biological Chains	30
5.1.2	Bi-Articular Muscles	30
5.1.3	Co-Contraction and Joint Stiffness	32
5.2	The Mass Matrix of the Human Body	32
6	Inverse Problems in Biomechanics	34
6.1	The Inverse Dynamics Pipeline	34
6.1.1	Inverse Kinematics	35
6.1.2	Bottom-Up vs. Top-Down Inverse Dynamics	36
6.2	The Muscle Redundancy Problem	36
6.2.1	Static Optimization	36
6.2.2	Computed Muscle Control (CMC)	36
6.3	Parameter Estimation	37
6.3.1	Bayesian Framework	37
7	Experimental Methods	38
7.1	Motion Capture Systems	38
7.1.1	Marker-Based Systems	38
7.1.2	Markerless Systems	39
7.1.3	Soft Tissue Artifact	40
7.1.4	Low-Pass Filter Cutoff Selection	40
7.2	Electromyography (EMG)	41
7.2.1	EMG Processing	41
7.3	Force Plates	41
7.4	Inertial Measurement Units (IMUs)	42
7.5	Data Synchronization	42
8	Inference on Biological Systems	43
8.1	Parameter Identification From Motion Data	43
8.1.1	Maximum Likelihood Estimation	43
8.1.2	Bayesian Estimation With Physical Constraints	44
8.2	System Identification for Biological Systems	45
8.3	Machine Learning Approaches	45

9	Deformable Bodies and Continuum Mechanics	47
9.1	When Rigid-Body Assumptions Fail	47
9.2	Beam Models for Long Bones and Shafts	47
9.2.1	Modal Analysis	48
9.2.2	Golf Club Shaft Flexibility	48
9.3	Finite Element Methods (Overview)	50
10	Applications of Control Theory to Biological Systems	51
10.1	Forward Dynamics Simulation	51
10.2	Predictive Simulation	53
10.3	Prosthetics and Exoskeleton Control	54
10.4	Sport Biomechanics Applications	54
10.4.1	The Golf Swing as a Biomechanical System	54
10.5	Synthesis: From Theory to Application	54

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Joint configuration vector (generalized coordinates).

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Joint velocity vector (generalized velocities).

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Joint acceleration vector.

$\mathbf{x} \in \mathbb{R}^{n_x}$ System state vector; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ for mechanical systems.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces/torques acting on the joints.

$t \in \mathbb{R}$ Time.

Inertia and Dynamics

$\mathbf{M}(\mathbf{q})$ Mass (inertia) matrix, symmetric positive definite.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ Coriolis and centrifugal force matrix.

$\mathbf{g}(\mathbf{q})$ Gravity vector.

$\mathbf{J}(\mathbf{q})$ Jacobian matrix relating joint velocities to task-space velocities ($\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$).

Control and Optimization

$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}$ Local Jacobian matrix of the state dynamics.

$\mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}$ Local Jacobian matrix of the control input.

$V(\mathbf{x})$ or $\mathcal{V}$ Value function or Lyapunov function.

$\mathbf{Q}$ State penalty matrix in LQR/DDP.

R Control penalty matrix in LQR/DDP.

K Feedback gain matrix ($\delta \mathbf{u} = -\mathbf{K}\delta \mathbf{x}$).

γ A trajectory or flow, mapping time and initial conditions to states.

Affine Control and Counterfactuals

$f(\mathbf{x})$ Drift vector field: complete autonomous evolution of the declared effective plant when $\mathbf{u} = \mathbf{0}$.

$G(\mathbf{x})$ Input (control) matrix field mapping control inputs to state derivatives.

$\dot{\mathbf{x}} = f(\mathbf{x}) + G(\mathbf{x})\mathbf{u}$ Control-affine system equations of motion.

$\mathbf{x}^{\text{ZTCF}}(t)$ Forward Zero-Torque Counterfactual trajectory under $\mathbf{u}(t) \equiv \mathbf{0}$.

$\mathbf{a}_{\text{ZVCF}}(\mathbf{q}, \mathbf{z})$ Instantaneous Zero-Velocity Counterfactual acceleration with velocity and declared control set to zero.

$\text{DCR}_{W, \mathcal{U}}(\mathbf{x})$ Drift-Control Ratio in a declared acceleration or task-projected space: $\frac{\|W \mathbf{a}_{\text{drift}}\|_2}{\sup_{\mathbf{u} \in \mathcal{U}} \|W B_a \mathbf{u}\|_2 + \varepsilon}$.

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

$\mathbf{F}_{\text{muscle}}$ Force generated by muscles.

$\mathbf{W}$ Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

How Biology Differs From Engineering

The golf swing connects neural excitation, muscle and tendon dynamics, joint motion, ground contact, and club deformation. A model can represent selected parts of this chain accurately while leaving others unresolved. Reproducing clubhead motion does not, by itself, establish which muscles generated it; reproducing net joint moments does not establish local tissue stress. This chapter develops the assumptions needed for a particular question and the corresponding tests of those assumptions.

The mathematical tools of the preceding volumes remain useful. Their application depends on the chosen state, input, constraints, and output. Biology changes those choices; it does not suspend mechanics. Numerical examples in this chapter are manufactured checks of equations, not measured human parameters or recommendations for a golfer.

1.1 The Rigid-Body Assumption and Its Limits

A rigid body has a fixed mass distribution in its own frame. A revolute joint permits one relative rotation and constrains the other five relative motions. Both are idealizations useful in biological and engineered systems. Bones deform, but their deformation need not be resolved for every movement question. Engineered shafts, transmissions, and sensors can also have compliance, backlash, noise, delay, and temperature dependence. The relevant distinction is between a declared approximation and the behavior it must explain.

For a golf model, declare the segment inventory, coordinate frames, joint constraints, and external contacts before selecting actuators. A shoulder rotation model may suffice for an endpoint-kinematics question while being inadequate for joint-contact loading. A hand-club constraint can transmit forces and moments even when its constrained coordinates are not actuated. Calling an anatomical joint “six degree of freedom” also requires care: relative pose has six components, but constraints and tissue mechanics may couple or strongly resist some motions. Coordinate count is not a count of independently controllable directions.

Biology vs. Engineering 1.1. Modeling Assumptions to Test

Structure

Choose rigid segments, compliant modes, or a continuum description according to the deformation that affects the target output.

Actuation

Specify actuator states and force, velocity, power, delay, and saturation limits where they matter.

Sensing

Identify observed quantities, error, sampling, filtering, and latency. A reconstructed state is not itself a sensor reading.

Adaptation

State the time horizon over which parameters are fixed. Fatigue, learning, and material changes require additional hypotheses.

Redundancy

Establish the rank of the relevant map and its feasible domain before claiming alternative solutions or compensation.

Motor torque can depend on speed, current dynamics, and thermal limits; muscle force can depend on activation, fibre length, velocity, and history. An ideal torque source deliberately suppresses these dependencies. It can be a useful first model of either actuator, provided conclusions remain about net loads and motion. A richer biological model should earn its added complexity through improved, independently tested predictions.

1.2 Compliance, Timescales, and the Chosen Output

1.2.1 Adding Deformation Requires Adding Dynamics

A free rigid club has six pose coordinates locally, not one grip angle. A one-angle club is a constrained planar abstraction. Flexible modes add coordinates to the selected rigid model. Write $\mathbf{z} = (\mathbf{q}, \boldsymbol{\eta})$, where $\mathbf{q} \in \mathbb{R}^n$ describes the rigid coordinates and $\boldsymbol{\eta} \in \mathbb{R}^r$ the retained deformation amplitudes. The selector $E = [0_{r \times n} \ I_r]$ gives $E\mathbf{z} = \boldsymbol{\eta}$. In a local chart with $\dot{\mathbf{z}}$ as its generalized velocity, an illustrative model is

$$M(\mathbf{z})\ddot{\mathbf{z}} + h(\mathbf{z}, \dot{\mathbf{z}}) + E^T(K\boldsymbol{\eta} + D\dot{\boldsymbol{\eta}}) = B(\mathbf{z})\boldsymbol{\tau} + J_c(\mathbf{z})^T\boldsymbol{\lambda}. \quad (1.1)$$

All terms are generalized forces in the same augmented coordinate space. M includes rigid–flexible inertial coupling; h contains remaining inertial and gravitational terms. Constant K, D describe a local linear elastic and viscous approximation, not arbitrary tissue behavior. B maps actuator loads $\boldsymbol{\tau}$ into the augmented coordinates, while J_c is the constraint Jacobian and $\boldsymbol{\lambda}$ its conjugate reaction loads. Contact equations complete the model. Adding an unprojected modal spring force to a rigid-body equation would omit both dimensions and coupling.

The shaft’s relevance depends on the output: small deformation can matter for club-face orientation while contributing little to whole-body motion. Ground contact and grip loads determine modal excitation; controller bandwidth alone does not describe that loading spectrum.

1.2.2 A Frequency Comparison With a Checkable Meaning

For a linear mode, let

$$m\ddot{\eta} + d\dot{\eta} + k\eta = F(t), \quad \omega_n = \sqrt{k/m}, \quad \zeta = \frac{d}{2\sqrt{km}}.$$

For a harmonic load of angular frequency Ω , steady-state displacement per unit force is

$$H(\Omega) = \frac{1}{k - m\Omega^2 + i d \Omega}, \quad kH(\Omega) = \frac{1}{1 - (\Omega/\omega_n)^2 + 2i\zeta\Omega/\omega_n}.$$

At low frequency, $H \simeq 1/k$: quasi-static compliance can still produce important deflection. Near resonance, damping and coupling matter. Far above the mode's frequency, inertia dominates the forced response; that does not establish rigidity during a transient with initial deformation and broadband loads.

Biomechanical Principle 1.1. The Timescale Separation Principle

Compare the relevant load spectrum and initial transients with modal frequencies and damping, then propagate omitted deformation to the output. Quasi-static reduction requires negligible dynamic error over that spectrum; a rigid approximation additionally requires negligible deformation error for the task. Neither follows from controller bandwidth alone.

For a manufactured translational mode with $m = 1$ kg, $k = 100$ N/m, and $d = 1$ N s/m, $\omega_n = 10$ rad/s and $\zeta = 0.05$. At $\Omega = 1, 10, 100$ rad/s, the amplitude relative to static compliance, $|kH|$, is approximately 1.010, 10, and 0.0101. The high-frequency response is nearly opposite in phase to the force. A constant static gain cannot reproduce all three regimes. These are illustrative parameters, not measurements of a golf shaft.

1.3 From Excitation to Joint Power

1.3.1 Force Definitions and Units

A common Hill-type idealization separates activation, normalized active force–length and force–velocity curves, and parallel passive force [18]. With positive fibre velocity denoting lengthening, write the illustrative undamped fibre force as

$$F_M = F_0 \left[a f_L(\ell_M/\ell_0) f_V(\dot{\ell}_M/v_0) + f_{PE}(\ell_M/\ell_0) \right]. \quad (1.2)$$

ℓ_M is fibre length, ℓ_0 its optimal reference length, and $a \in [0, 1]$ the activation state. F_0 is the maximum active isometric fibre-force scale at optimal length; the curve values are dimensionless. $v_0 > 0$ is a declared velocity scale, not a universal human shortening speed. The parallel-elastic term f_{PE} is scaled by the same outside factor F_0 . Other models add damping or history dependence and use different normalizations.

Fibre force is not automatically tendon force. In a massless, ideal pennated equilibrium model, $F_T = F_M \cos \alpha$, where α is the pennation angle between fibres and tendon. This relation requires a tendon constitutive law and compatible fibre–tendon geometry. A compliant tendon can change length while the whole muscletendon path length changes little. Joint angular velocity therefore does not by itself determine fibre shortening velocity. Model-specific equilibrium, slack, and regularization conditions must be checked before solving for fibre velocity.

1.3.2 Moment Arms Must Preserve Power

For differentiable, time-independent musculotendon routing $\ell_{MT}(\mathbf{q})$, define

$$R_{ji} = -\frac{\partial \ell_{MT,i}}{\partial q_j}, \quad \boldsymbol{\tau}_M = R(\mathbf{q})\mathbf{F}_T. \quad (1.3)$$

For rotational coordinates, R_{ji} has length units and the conjugate load is a moment. Translational coordinates instead have force as the conjugate load. Virtual work gives the sign-sensitive check

$$\dot{\ell}_{MT} = -R^T \dot{\mathbf{q}}, \quad \boldsymbol{\tau}_M^T \dot{\mathbf{q}} = -\mathbf{F}_T^T \dot{\ell}_{MT}.$$

A positive tensile force delivers positive skeletal power when its path shortens. Fibre power can differ from path power because tendon energy storage and other internal processes intervene. For constrained coordinates or speeds distinct from $\dot{\mathbf{q}}$, apply the velocity map before comparing power.

For a numerical check, take rotational coordinates and

$$R = \begin{bmatrix} 0.03 & -0.03 & 0 \\ 0 & 0.02 & -0.02 \end{bmatrix} \text{ m}, \quad \mathbf{F}_T = \begin{bmatrix} 100 \\ 80 \\ 60 \end{bmatrix} \text{ N}, \quad \dot{\mathbf{q}} = \begin{bmatrix} 2 \\ -1 \end{bmatrix} \text{ s}^{-1}.$$

Then $\boldsymbol{\tau}_M = (0.6, 0.4)^T$ N m and $\dot{\ell}_{MT} = (-0.06, 0.08, -0.02)^T$ m/s. Both expressions for skeletal power give 0.8 W. A moment-arm sign error can look plausible in a torque plot while failing energy accounting immediately.

Biomechanical Principle 1.2. Force-Velocity Asymmetry in Multi-Joint Tasks

Force capacity depends on each muscle's fibre state, activation, parameters, and constitutive curves. A biarticular muscle couples multiple joint motions through its routing and tendon dynamics. Evaluate those states before classifying its contraction or constructing the feasible joint-load set; joint rotation alone does not specify eccentric or concentric fibre contraction.

In a downswing, the same visible wrist motion can be compatible with different tendon strains, activation histories, and antagonist forces. Muscle modeling makes those alternatives calculable under assumptions; it does not make video sufficient to identify which alternative occurred.

1.4 Two Different Kinds of Redundancy

Let the task be $\mathbf{y} = \phi(\mathbf{q})$, with Jacobian $J_y = \partial\phi/\partial\mathbf{q}$. For n independent local configuration coordinates, infinitesimal motions preserving $\mathbf{y}$ satisfy $J_y \delta\mathbf{q} = 0$, with dimension $n - \text{rank } J_y$. A three-joint planar arm controlling a 2D endpoint position has a one-dimensional local nullspace only where the Jacobian has rank two. At singular configurations the rank changes. A finite displacement requires following the nonlinear level set, not extrapolating one tangent direction indefinitely.

Muscle-force redundancy concerns a different map. At fixed geometry, an instantaneous force-allocation model solves $R\mathbf{F}_T = \boldsymbol{\tau}_{\text{req}}$, with a linear equality solution space of

dimension $p - \text{rank } R$ when nonempty, for p muscle paths. Nonnegative tensile forces and state-dependent capacities restrict this space. In the dimensionless algebraic example

$$R = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \quad \tau_{\text{req}} = \begin{bmatrix} 3 \\ 3 \end{bmatrix},$$

all equality solutions are $(3 - t, 3 - t, t)^T$. Bounds $0 \leq F_i \leq 3$ leave $0 \leq t \leq 3$. Other loads or capacities can leave a point or no solution. This is an allocation example, not a human moment-arm model. It shows why a nullspace count cannot guarantee compensation for a fatigued muscle. These candidate forces must still satisfy muscle dynamics: if every internal state is already fixed, a constitutive model may fix tendon force as well. An allocation equality alone does not supply a realizable excitation history.

Even equal nullity does not identify equal physical freedom. The task map $J_y = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$ has null vector $(1, -1, 1)^T$, whereas the example R has null vector $(-1, -1, 1)^T$. Their domains also represent different quantities: configuration changes and muscle forces. Matching hand position does not guarantee matching velocity, joint loading, energy use, or future motion.

The uncontrolled manifold hypothesis tests whether variability is preferentially organized along directions preserving a selected task variable [22]. Its original sit-to-stand analysis supports a task-dependent interpretation of joint variability. It is not a direct measurement of a neural projection operator or proof that all task-null directions are left uncontrolled. Coordinate scaling, task choice, noise, and dynamics influence interpretation. In golf, variability preserving clubhead position at one instant can still change face orientation, impact timing, or the subsequent response to a perturbation.

1.5 Choosing the Model by the Question

1.5.1 Rigid Segments With Generalized-Load Actuators

This model examines motion, power, momentum, and net loads without resolving individual muscles. Inverse dynamics requires kinematics, inertial parameters, external loads, and constraints; it does not identify individual muscle forces from motion alone. Add actuator bounds when testing trajectory feasibility. An unconstrained optimal torque history is a mathematical result, not evidence that a golfer can execute it.

For clubhead speed, this level can clarify how mass distribution, segment motion, and external forces interact. A sensitivity calculation can ask whether plausible parameter changes or shaft deformation alter the conclusion. If they do, examine the omitted physics before interpreting the result as a mechanism of a human swing.

1.5.2 Rigid Segments With Muscle and Tendon States

This adds routing, activation, and constitutive assumptions. A forward simulation requires excitations and initial states. Estimating excitation from observed motion requires further data, a control hypothesis, or an optimization criterion. Multiple solutions may fit the same motion. Report residual forces, reserve actuators, parameter sensitivity, and objective sensitivity rather than presenting one optimum as an observed neural strategy.

OpenSim supports musculoskeletal modeling and simulation; a typical model combines rigid bodies, joints, and force elements [8]. Such a model is not automatically a finite-element

model of deforming bone or soft tissue. Joint reactions are resultant loads conditional on modeled actuators and constraints, not local stress fields.

1.5.3 Deformation or Tissue Stress as an Output

Flexible modes, continuum models, or coupled multibody–finite-element models resolve deformation when the question requires it. Local stress requires geometry, constitutive properties, boundary conditions, and loading; numerical resolution and constitutive validation are separate checks. A resultant force and moment constrain integrals of traction without uniquely determining its spatial distribution. A calculated stress field is not an injury probability without a validated injury or failure model.

These choices are not a universal fidelity ranking. Extra parameters can increase uncertainty when measurements cannot identify them. Real-time feasibility depends on model, algorithm, hardware, accuracy, and update rate; a fixed degree-of-freedom threshold cannot select the method. State an output tolerance, compare simpler and richer models against independent observations, and retain the least complexity that supports the claim.

1.6 Mass Properties Need Accounting and Frames

1.6.1 A Complete Segment Inventory

Anthropometric coefficients require a population, segmentation scheme, landmarks, and parameter conventions. De Leva’s study adjusts landmark references of Zatsiorsky–Seluyanov segment parameters [7]; it is not a generic update of Dempster cadaver coefficients. Mixing incompatible segmentations can double-count or omit mass. Scaling population estimates also does not make them subject-specific measurements.

An inconsistent inventory illustrates the danger: central fractions 0.081 and 0.497 combined with six unilateral fractions 0.0271, 0.0162, 0.0061, 0.1416, 0.0433, and 0.0137. Their one-row sum is 0.826; counting both sides gives 1.074. Neither accounts for one body. Renormalizing conceals the provenance problem. No empirical replacement table is asserted here without verification of its source, landmarks, and segment definitions.

For a synthetic accounting exercise, use central fractions 0.10, 0.40 and six paired fractions (0.05, 0.03, 0.02, 0.09, 0.04, 0.02) per side. Their bilateral sum is one. For 75 kg, the central masses are 7.5, 30 kg and each side has (3.75, 2.25, 1.50, 6.75, 3.00, 1.50) kg. This tests bookkeeping only; it provides neither human lengths nor centres of mass or inertias.

1.6.2 Centre of Mass and Spatial Inertia

Let a segment’s local centre of mass be $\mathbf{c}_i$, its local-to-world rotation matrix $Q_i \in SO(3)$, and its world origin $\mathbf{p}_i$. Then

$$\mathbf{r}_i = \mathbf{p}_i + Q_i \mathbf{c}_i, \quad \mathbf{r}_{COM} = \frac{\sum_i m_i \mathbf{r}_i}{\sum_i m_i}.$$

Body mass and height alone cannot specify these world positions. A pose and kinematic model are needed, with separate left and right instances. For masses 2, 1 kg at (0.2, 0, 0), (0.4, 0.15, 0) m, respectively, the centre is (4/15, 0.05, 0) m. Translating both positions translates the centre by the same amount, a useful frame-consistency check.

A scalar $I = m(\rho L)^2$ describes inertia about a specified axis when ρL is its radius of gyration. General 3D dynamics needs a symmetric tensor I_C about the centre of mass in declared axes. For an angular-first twist $V = (\boldsymbol{\omega}, \mathbf{v}_O)$, all expressed in one frame, let $\mathbf{c}$ point from origin O to the centre and $C = [\mathbf{c}]_\times$. Then

$$\mathcal{I}_O = \begin{bmatrix} I_C - mCC & mC \\ -mC & mI_3 \end{bmatrix}.$$

The signs follow from $\mathbf{v}_C = \mathbf{v}_O + \boldsymbol{\omega} \times \mathbf{c} = \mathbf{v}_O - C\boldsymbol{\omega}$, giving

$$\frac{1}{2}V^T \mathcal{I}_O V = \frac{1}{2}\boldsymbol{\omega}^T I_C \boldsymbol{\omega} + \frac{1}{2}m\|\mathbf{v}_O + \boldsymbol{\omega} \times \mathbf{c}\|^2.$$

Positive mass and positive-definite I_C make this form positive definite. Physical inertia tensors also satisfy principal moment triangle inequalities; positive eigenvalues alone are not a complete realizability test. Point or ideal line masses can have singular rotational inertia. Declare which case an implementation accepts.

1.7 When the Control-Affine Framework Applies

1.7.1 Choose the Input Before Splitting Drift and Control

Torque-driven and excitation-driven models describe different interventions. For a smooth fixed-contact model in independent coordinates, let $\mathbf{s}$ collect muscle states, with composite state $\mathbf{x} = (\mathbf{q}, \mathbf{v}, \mathbf{s})$. Assume M is invertible on the domain under analysis. A possible description is

$$\dot{\mathbf{q}} = N(\mathbf{q})\mathbf{v}, \quad M(\mathbf{q})\dot{\mathbf{v}} = \mathbf{f}_{\text{mech}}(\mathbf{q}, \mathbf{v}, \mathbf{s}), \quad \dot{\mathbf{s}} = h_s(\mathbf{q}, \mathbf{v}, \mathbf{s}, \mathbf{u}).$$

N accounts for representations whose coordinate derivatives differ from generalized speeds. Reactions must be solved consistently or eliminated on the constraint manifold. Here excitation acts through muscle states, with no separate direct excitation term in mechanics. Input affinity depends on h_s and any other input-dependent equations.

Biomechanical Principle 1.3. The Extended Affine Structure

A declared model is control-affine in its chosen input if

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{G}(\mathbf{x})\mathbf{u}. \quad (1.4)$$

Nonlinear state dependence in $\mathbf{f}, \mathbf{G}$ is permitted. For example, $\dot{a} = (u - a)/T$ with constant $T > 0$ is affine in excitation. If remaining muscle-state equations depend on activation and mechanics but not directly on excitation, their nonlinearity does not invalidate this structure. Establish these assumptions for the selected model; do not infer them from the word “musculoskeletal.”

Consider instead the branch-dependent law documented for OpenSim’s [first-order activation model](#):

$$\dot{a} = \frac{u - a}{T(u, a)}, \quad T(u, a) = \begin{cases} T_{\text{act}}(0.5 + 1.5a), & u > a, \\ T_{\text{deact}}/(0.5 + 1.5a), & u \leq a. \end{cases}$$

For a manufactured evaluation without an activation floor, choose $a = 0.5$, $T_{act} = 0.01$ s, and $T_{deact} = 0.04$ s. The rates at $u = 0, 0.5, 1$ are $-15.625, 0, 40$ s⁻¹. An affine input map would give the midpoint rate $(-15.625 + 40)/2 = 12.1875$ s⁻¹, not zero. Each branch is affine in u for fixed state, but the full map is not. The rate is continuous at $u = a$, with generally unequal one-sided slopes. Activation floors and clamping in an implementation need their own treatment; this arithmetic does not reproduce every software variant.

Remark 1.1. Dimensional Expansion of the State Space

For n independent coordinates, p activations, and p independent fibre length states, a local state dimension is $2n + 2p$. A rigid-tendon model may omit fibre-length states; fatigue or history states increase the count. Algebraic constraints and redundant pose representations change the distinction between stored variables and independent states. Derive dimension from the equations rather than a universal number of states per muscle.

Lie-bracket calculations require appropriate smooth vector fields and a specified admissible input set. Contraction arguments also require a suitable metric, invariant domain, and the theorem's differential conditions. Input bounds, contact transitions, and nonsmooth force laws can limit conclusions. Smooth analysis within a contact mode is not a proof across impacts or changes in contact. Dimensionality is a computational challenge; validity of the mathematical hypotheses comes first.

Zero excitation does not instantly erase activation or tendon strain. With constant T , activation subsequently decays as $a(t) = a(0)e^{-t/T}$ while other states evolve. Zero-input drift can thus include force associated with prior excitation. It is not synonymous with anatomically passive force, zero joint torque, or zero power. A golf counterfactual must specify what is removed and which initial states, contacts, and feedback rules are retained.

1.8 Connecting the Parts of the Swing Argument

The choices form a chain of inference. Kinematics and external loads, with mass properties and contacts, constrain net dynamics. Routing and constitutive assumptions map candidate muscle states into those loads. Activation dynamics connect states to candidate excitations. Compliance changes both transmission and the output being controlled. Redundancy leaves alternatives that an objective or further measurement may restrict. Every added explanation introduces a corresponding test.

A claim that a golfer increases clubhead speed through a particular muscle action requires more than a faster clubhead trajectory. Test whether the conclusion persists across plausible inertial parameters, grip constraints, tendon properties, allocation objectives, and shaft models. Separate an instantaneous power identity from a finite-time causal intervention: removing one torque or excitation changes later states and may change loads supplied by feedback. Report the intervention and its uncertainty alongside the predicted effect.

The resulting explanation should distinguish measured quantities, mechanical calculations, biologically conditioned inferences, and hypotheses about control or coaching. That distinction lets a simple model provide strong insight while identifying what remains unresolved about the swing.

Exercises With Verification Targets

1. **Mass Accounting.** Reproduce both erroneous inventory sums and the synthetic 75-kg inventory. Explain why a unit sum cannot validate anatomical provenance or supply missing centre-of-mass locations.
2. **Compliance.** Reproduce the three values of $|kH|$ above. A hypothetical 5-Hz mode completes $5(0.25) = 1.25$ cycles in 0.25 s. Explain why this count supplies neither excitation amplitude nor output error and cannot decide whether shaft deformation is negligible.
3. **Power.** Verify the 0.8-W example by both routes. Identify the energy-storage terms needed to equate skeletal and contractile-element power for a compliant tendon.
4. **Redundancy.** Derive both null vectors and the interval $0 \leq t \leq 3$. For three joints and six muscle paths, state the ranks needed for task nullity one and force nullity three. Explain why neither number counts identifiable neural excitation histories.
5. **Mass Frames.** Reproduce the two-mass centre of mass and its translation covariance. With upward y and gravity magnitude g , compute $U = \sum_i m_i g y_i = 0.15g$. State the reference level and why mass fractions alone cannot determine potential energy.
6. **Spatial Inertia.** Use $m = 2$ kg and

$$\mathbf{c} = (0.1, -0.2, 0.3) \text{ m}, \quad I_C = \text{diag}(0.02, 0.03, 0.04) \text{ kg m}^2.$$

Check symmetry, positivity, principal moment inequalities, and both energy expressions for $\boldsymbol{\omega} = (1.5, -2, 0.5) \text{ s}^{-1}$ and $\mathbf{v}_O = (-1.2, 0.8, -0.4) \text{ m/s}$. Nonzero translation tests the cross-block signs.

7. **Input Affinity.** Compare the branch-dependent example with the constant- T model, $T = 0.02$ s, giving rates $(-25, 0, 25) \text{ s}^{-1}$ at the same inputs and activation. Explain why nonlinear activation dependence alone does not disprove input affinity, and why zero excitation is not an instantaneous relaxed state.
8. **Model Selection.** Specify separate evidence requirements for clubface pose, net wrist moment, individual tendon force, and local stress. For a force-velocity calculation, identify the missing curve, normalization, fibre state, activation, and tendon assumptions before assigning an eccentric-to-isometric force ratio.

Chapter 2

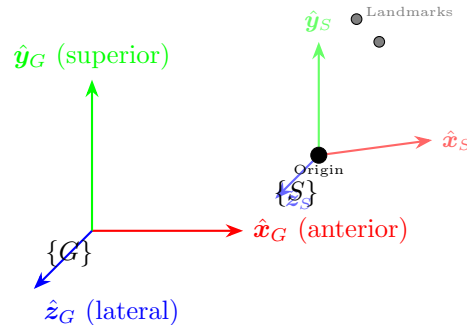
Musculoskeletal Modeling Conventions

In biomechanics, the first source of error is not the mathematics but the coordinate system.

— ISB Standardization Committee

2.1 Anatomical Reference Frames

Biomechanical analysis requires a standardized language for describing body position and motion. The International Society of Biomechanics (ISB) has published recommendations for joint coordinate systems [28, 29] that serve as the international standard.



2.1.1 The Anatomical Position

All references to body position begin from the **anatomical position**: the subject stands upright, feet together, arms at sides, palms facing forward. This defines the *zero configuration* from which all joint angles are measured.

Definition 2.1. Global Reference Frame

The global (laboratory) reference frame $\{G\}$ is defined as:

- $\hat{x}_G$: anterior (forward)
- $\hat{y}_G$: superior (upward)

- $\hat{\mathbf{z}}_G$: lateral right

This is a right-handed coordinate system. Note that different communities use different conventions: biomechanics typically uses *Y*-up, while robotics often uses *Z*-up. **Always verify the convention before combining data from different sources.**

2.1.2 Segment Reference Frames

Each body segment has a local reference frame attached to it, defined by bony landmarks that can be palpated or identified from medical imaging. The ISB recommendations specify these landmarks for each segment [28, 29].

Example 2.1. The Thigh Reference Frame

The ISB-recommended thigh reference frame $\{T\}$ is defined by:

- **Origin**: midpoint between medial and lateral femoral epicondyles
- $\hat{\mathbf{y}}_T$: from origin to hip joint center (proximal direction)
- $\hat{\mathbf{z}}_T$: perpendicular to the plane formed by the two epicondyles and the hip center, pointing laterally
- $\hat{\mathbf{x}}_T$: cross product $\hat{\mathbf{y}}_T \times \hat{\mathbf{z}}_T$ (anterior direction)

The hip joint center is estimated from the pelvis landmarks using regression equations (e.g., [12]).

2.1.3 Connection to Screw Theory

In the language of Volume 0, each segment frame $\{S_i\}$ is related to the global frame $\{G\}$ by a homogeneous transformation matrix $T_{GS_i} \in \text{SE}(3)$:

$$T_{GS_i} = \begin{bmatrix} R_{GS_i} & \mathbf{p}_{GS_i} \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad (2.1)$$

where $R_{GS_i} \in \text{SO}(3)$ is the rotation matrix and $\mathbf{p}_{GS_i} \in \mathbb{R}^3$ is the position of the segment origin in the global frame.

The *relative configuration* of segment j with respect to proximal segment i is:

$$T_{S_i S_j} = T_{GS_i}^{-1} T_{GS_j}. \quad (2.2)$$

Joint angles are extracted from this relative transformation.

2.2 Joint Coordinate Systems

The ISB recommends decomposing the relative rotation $R_{S_i S_j}$ into three angles using a specific Euler angle sequence for each joint. The choice of sequence is *not arbitrary*—it is

selected to align with the clinical definitions of flexion-extension, abduction-adduction, and internal-external rotation.

2.2.1 The Grood–Suntay Convention for the Knee

The knee joint coordinate system, originally proposed by Grood and Suntay (1983) [10], decomposes knee motion into:

1. **Flexion-Extension** (α): rotation about the femoral medio-lateral axis (the “fixed” axis of the femur)
2. **Abduction-Adduction** (β): rotation about the “floating” axis (perpendicular to both the femoral and tibial body-fixed axes)
3. **Internal-External Rotation** (γ): rotation about the tibial longitudinal axis (the “body-fixed” axis of the tibia)

This is a ZXY Euler angle sequence when expressed in the Grood–Suntay convention. The key insight is that the floating axis is neither fixed in the femur nor in the tibia—it is defined by the cross product of the other two axes.

```

1 import numpy as np
2 from typing import Tuple
3
4 def grood_suntay_knee_angles(
5     R_femur: np.ndarray,
6     R_tibia: np.ndarray
7 ) -> Tuple[float, float, float]:
8     """Compute knee joint angles using Grood-Suntay convention.
9
10    Parameters
11    -----
12    R_femur : np.ndarray, shape (3, 3)
13        Rotation matrix of femur segment frame in global coordinates.
14    R_tibia : np.ndarray, shape (3, 3)
15        Rotation matrix of tibia segment frame in global coordinates.
16
17    Returns
18    -----
19    Tuple[float, float, float]
20        (flexion_deg, adduction_deg, internal_rotation_deg)
21
22    References
23    -----
24    Grood, E.S. and Suntay, W.J. (1983). A joint coordinate system
25    for the clinical description of three-dimensional motions.
26    J. Biomechanical Engineering, 105(2), 136-144.
27    """
28    # Femoral medio-lateral axis (e1) - flexion axis
29    e1 = R_femur[:, 2] # lateral axis of femur
30
31    # Tibial longitudinal axis (e3) - rotation axis
32    e3 = R_tibia[:, 1] # proximal axis of tibia
33
34    # Floating axis (e2) - perpendicular to both
35    e2 = np.cross(e3, e1)
36    e2 = e2 / np.linalg.norm(e2)
37

```

```

38 # Flexion-extension angle
39 # Angle between femur anterior axis and floating axis
40 femur_anterior = R_femur[:, 0]
41 flexion = np.arctan2(
42     np.dot(e2, R_femur[:, 1]),
43     np.dot(e2, femur_anterior)
44 )
45
46 # Abduction-adduction angle
47 # Angle between e1 and e3 minus 90 degrees
48 sin_abd = -np.dot(e1, e3)
49 cos_abd = np.linalg.norm(np.cross(e1, e3))
50 adduction = np.arcsin(np.clip(sin_abd, -1.0, 1.0))
51
52 # Internal-external rotation
53 # Angle between floating axis and tibia anterior axis
54 tibia_anterior = R_tibia[:, 0]
55 internal_rot = np.arctan2(
56     np.dot(e2, R_tibia[:, 2]),
57     np.dot(e2, tibia_anterior)
58 )
59
60 return (
61     np.degrees(flexion),
62     np.degrees(adduction),
63     np.degrees(internal_rot)
64 )

```

Listing 2.1: Computing knee joint angles using the Grood-Suntay convention.

2.2.2 Multi-Representation Joint Angles

Following the multi-representation philosophy of this series, we express joint rotations in all four standard representations:

1. Euler Angles (ISB Convention):

$$R_{ij} = R_z(\alpha) R_x(\beta) R_y(\gamma), \quad (2.3)$$

where α, β, γ are the clinical angles (flexion, adduction, rotation).

2. Rotation Matrix (SO(3)):

$$R_{ij} = \exp([\omega]_{\times} \theta) \in \text{SO}(3), \quad (2.4)$$

which avoids gimbal lock but requires 9 parameters with 6 constraints.

3. Unit Quaternion:

$$\mathbf{q} = \left(\cos \frac{\theta}{2}, \boldsymbol{\omega} \sin \frac{\theta}{2} \right), \quad \|\mathbf{q}\| = 1, \quad (2.5)$$

which is singularity-free and computationally efficient.

4. Screw Axis (Primary):

$$T_{ij} = e^{[S]\theta} \in \text{SE}(3), \quad S = (\boldsymbol{\omega}, \mathbf{v}) \in \mathbb{R}^6, \quad (2.6)$$

which captures the coupled rotation-translation of biological joints naturally.

Biology vs. Engineering 2.1. Why Screw Axis Theory Is Natural for Biological Joints

Unlike engineering revolute joints (pure rotation about a fixed axis), biological joints exhibit *coupled* rotation and translation. The knee slides anteriorly as it flexes; the glenohumeral joint translates inferiorly during abduction. Screw axis theory captures this coupling naturally: every rigid-body motion is a rotation about and translation along a screw axis. The instantaneous screw axis (ISA) of a biological joint moves through space as the joint moves, unlike the fixed axis of an ideal revolute. This is precisely why Park and Lynch's *Modern Robotics* framework is preferred: it handles coupled rotation-translation without special cases.

2.3 Body Segment Parameters

2.3.1 Regression Equations

The most widely used body segment parameter (BSP) data comes from:

1. **Dempster (1955)**: cadaver study of 8 elderly males. Still widely cited but limited demographic range.
2. **Zatsiorsky & Seluyanov (1983, 1990)**: gamma-ray scanning of 100 living young males. More representative but uses different segment definitions.
3. **de Leva (1996)**: adjustment of Zatsiorsky data to standard segment definitions. The current gold standard for adult male and female BSP.
4. **Pavol et al. (2002)**: elderly adult data.
5. **Jensen (1986, 1989)**: pediatric data.

2.3.2 Computing the Mass Matrix

Given segment inertial properties, the mass matrix $M(\mathbf{q}) \in \mathbb{R}^{n \times n}$ of the multi-body system is computed using the composite rigid-body algorithm (Volume 0, Chapter 10). In the body-fixed frame of segment i , the spatial inertia is:

$$\mathcal{G}_i = \begin{bmatrix} I_i & m_i[\mathbf{c}_i]_{\times}^T \\ m_i[\mathbf{c}_i]_{\times} & m_i\mathbb{I}_3 \end{bmatrix} \in \mathbb{R}^{6 \times 6}, \quad (2.7)$$

where I_i is the 3×3 rotational inertia about the segment frame origin, m_i is the segment mass, $\mathbf{c}_i$ is the center of mass position in the segment frame, and $[\cdot]_{\times}$ denotes the skew-symmetric matrix.

Exercises

1. **Coordinate System Verification.** Verify that the Grood–Suntay knee joint coordinate system produces zero angles when the femur and tibia reference frames are aligned. Test with known rotations.

2. **Euler Angle Singularity.** For the ZXY Euler angle sequence used in the knee, identify the gimbal lock configuration. Is this configuration ever reached during normal gait? During deep squatting?
3. **Multi-Representation Conversion.** Write Python functions to convert between all four representations (Euler ZXY, rotation matrix, quaternion, screw axis) for a knee joint angle of 60° flexion, 5° adduction, 10° internal rotation.
4. **BSP Sensitivity.** Using the de Leva segment properties, compute the knee extension torque required to hold the lower leg horizontal (isometric). Repeat using the Dempster data. What is the percentage difference? Discuss the implications for inverse dynamics accuracy.
5. **(Programming.)** Implement the spatial inertia matrix computation for a 3-segment leg model (thigh, shank, foot). Verify positive-definiteness.
6. **(Programming.)** Using motion capture data (or simulated data), compute knee joint angles from segment orientations using the Grood–Suntay code provided. Plot flexion angle over one gait cycle.

Chapter 3

Muscle Models

A muscle is not a motor. It is a spring with a memory, a damper with an agenda, and a force generator with a personality.

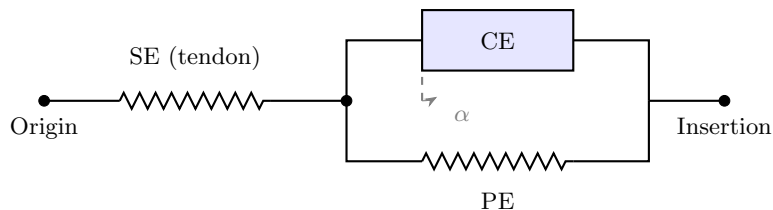
3.1 The Hill-Type Muscle Model

The Hill-type muscle model, originating from A.V. Hill's Nobel Prize-winning work on muscle thermodynamics and muscle energetics [13, 14], is the standard lumped-parameter model used in biomechanical simulation. It represents the musculotendon unit as a combination of contractile, elastic, and damping elements. Modern implementations include refinements by [23] and [18].

3.1.1 Model Architecture

The standard Hill-type musculotendon model consists of three elements:

1. **Contractile Element (CE)**: represents the active force-generating capacity of the muscle fibers. Its force depends on activation $a(t)$, fiber length ℓ^M , and fiber velocity $v^M = \dot{\ell}^M$.
2. **Parallel Elastic Element (PE)**: represents the passive elastic properties of the muscle belly (titin filaments, connective tissue). Produces force only when the muscle is stretched beyond its resting length.
3. **Series Elastic Element (SE)**: represents the tendon, which is in series with the contractile element. Modeled as a nonlinear spring with a slack length ℓ_T^s .



3.1.2 The Force-Length Relationship

The isometric (zero velocity) force of the contractile element depends on the normalized fiber length $\tilde{\ell} = \ell^M / \ell_0^M$:

Definition 3.1. Active Force-Length Curve

The normalized active force-length relationship is:

$$f_L(\tilde{\ell}) = \exp\left(-\frac{(\tilde{\ell} - 1)^2}{\gamma}\right), \quad (3.1)$$

where $\gamma \approx 0.45$ [23] controls the width of the curve. Maximum force ($f_L = 1$) occurs at optimal fiber length ($\tilde{\ell} = 1$). Force decreases for both shorter and longer fiber lengths, reaching near-zero at approximately $\tilde{\ell} = 0.5$ and $\tilde{\ell} = 1.5$.

A note on the numerical parameters. The values $\gamma = 0.45$, $A_f = 0.25$, $k^{PE} = 4.0$, and $\varepsilon_0 = 0.6$ used throughout this chapter are the default parameters recommended by Thelen [23] for the “generic” adult Hill-type muscle, and also serve as the baseline for the Millard–Uchida formulation [18]. They are calibrated against mid-thigh muscles of young adults and should be interpreted as a starting point, not as universal constants. Real muscles exhibit systematic variation with age, sex, training status, and fiber-type composition, and subject-specific tuning is almost always required for quantitative predictions.

The physical basis for the force-length relationship is the sliding filament theory [15]: at the optimal length, the maximum number of actin-myosin cross-bridges can form. At shorter lengths, filament overlap reduces the number of available binding sites. At longer lengths, the filaments pull apart.

```

1 import numpy as np
2 from dataclasses import dataclass
3 from typing import Tuple
4
5 @dataclass
6 class MuscleParameters:
7     """Parameters for a Hill-type musculotendon model."""
8     F0: float          # Maximum isometric force (N)
9     l0_M: float        # Optimal fiber length (m)
10    v0: float          # Maximum contraction velocity (m/s)
11    l_T_slack: float    # Tendon slack length (m)
12    alpha0: float       # Pennation angle at optimal length (rad)
13    # Defaults below are the Thelen (2003) generic-adult values.
14    # See: Thelen, D.G. (2003) "Adjustment of muscle mechanics
15    # model parameters to simulate dynamic contractions in older
16    # adults," J. Biomech. Eng. 125(1), 70-77 [Thelen2003].
17    gamma: float = 0.45 # Force-length curve width (Thelen 2003)
18    Af: float = 0.25    # Force-velocity shape factor (Thelen 2003)
19    kPE: float = 4.0    # Passive elastic stiffness (Thelen 2003)
20    e0: float = 0.6     # Passive strain at F0 (Thelen 2003)
21
22    @property
23    def v_max(self) -> float:
24        """Maximum shortening velocity in fiber lengths/s."""

```

```

25         return self.v0 / self.l0_M
26
27
28 def force_length_active(l_tilde: float, gamma: float = 0.45) -> float:
29     """Active force-length relationship.
30
31     Parameters
32     -----
33     l_tilde : float
34         Normalized fiber length (l_M / l0_M).
35     gamma : float
36         Width parameter of the Gaussian curve.
37
38     Returns
39     -----
40     float
41         Normalized active force (0 to 1).
42
43     References
44     -----
45     Thelen, D.G. (2003). Adjustment of muscle mechanics model
46     parameters to simulate dynamic contractions in older adults.
47     J. Biomechanical Engineering, 125(1), 70-77.
48     """
49     return np.exp(-((l_tilde - 1.0) ** 2) / gamma)
50
51
52 def force_length_passive(l_tilde: float, kPE: float = 4.0,
53                          e0: float = 0.6) -> float:
54     """Passive force-length relationship (exponential model).
55
56     Parameters
57     -----
58     l_tilde : float
59         Normalized fiber length.
60     kPE : float
61         Stiffness parameter.
62     e0 : float
63         Strain at which passive force equals F0.
64
65     Returns
66     -----
67     float
68         Normalized passive force.
69     """
70     if l_tilde <= 1.0:
71         return 0.0
72     return (np.exp(kPE * (l_tilde - 1.0) / e0 - kPE) - 1.0) / \
73            (np.exp(kPE) - 1.0)
74
75
76 def force_velocity(v_tilde: float, a: float, Af: float = 0.25,
77                   Flen: float = 1.4) -> float:
78     """Force-velocity relationship (Thelen 2003 model).
79
80     Parameters
81     -----
82     v_tilde : float
83         Normalized fiber velocity (v_M / v_max).
84         Negative = shortening (concentric).

```

```

85     a : float
86         Activation level (0 to 1).
87     Af : float
88         Shape factor for concentric portion.
89     Flen : float
90         Maximum normalized eccentric force.
91
92     Returns
93     -----
94     float
95         Normalized force multiplier.
96     """
97     if v_tilde <= 0:
98         # Concentric (shortening)
99         return (1.0 + v_tilde) / (1.0 - v_tilde / (Af * a + 0.01))
100     else:
101         # Eccentric (lengthening)
102         return (1.0 + v_tilde * Flen / (0.04 + a)) / \
103             (1.0 + v_tilde / (0.04 + a))
104
105
106 def muscle_force(
107     activation: float,
108     l_M: float,
109     v_M: float,
110     params: MuscleParameters
111 ) -> Tuple[float, float, float]:
112     """Compute total musculotendon force.
113
114     Parameters
115     -----
116     activation : float
117         Neural activation level (0 to 1).
118     l_M : float
119         Current muscle fiber length (m).
120     v_M : float
121         Current muscle fiber velocity (m/s, negative = shortening).
122     params : MuscleParameters
123         Muscle parameters.
124
125     Returns
126     -----
127     Tuple[float, float, float]
128         (total_force, active_force, passive_force) in Newtons.
129     """
130     # Normalize
131     l_tilde = l_M / params.l0_M
132     v_tilde = v_M / params.v0
133
134     # Force components
135     f_active = (activation
136                 * force_length_active(l_tilde, params.gamma)
137                 * force_velocity(v_tilde, activation, params.Af))
138     f_passive = force_length_passive(l_tilde, params.kPE, params.e0)
139
140     # Pennation angle varies with fiber length under the
141     # constant-thickness assumption:  $\sin(\alpha) = \sin(\alpha_0) * l_{0\_M} / l\_M$ 
142     sin_alpha = np.sin(params.alpha0) * params.l0_M / max(l_M, 1e-6)
143     cos_alpha = np.sqrt(max(0.0, 1.0 - sin_alpha ** 2))
144

```

```

145     # Total force along tendon
146     F_active = f_active * params.F0 * cos_alpha
147     F_passive = f_passive * params.F0 * cos_alpha
148     F_total = F_active + F_passive
149
150     return F_total, F_active, F_passive
151
152
153 # Example: Biceps brachii
154 biceps = MuscleParameters(
155     F0=624.3,          # N (Holzbaur et al. 2005, upper-extremity model)
156     l0_M=0.1157,      # m
157     v0=10 * 0.1157,   # 10 fiber lengths/s
158     l_T_slack=0.2723, # m
159     alpha0=0.0        # parallel-fibered
160 )
161
162 # Compute force at optimal length, half activation, isometric
163 F_total, F_active, F_passive = muscle_force(
164     activation=0.5,
165     l_M=biceps.l0_M, # optimal length
166     v_M=0.0,         # isometric
167     params=biceps
168 )
169 print(f"Biceps at 50% activation, optimal length, isometric:")
170 print(f"  Active force: {F_active:.1f} N")
171 print(f"  Passive force: {F_passive:.1f} N")
172 print(f"  Total force: {F_total:.1f} N")

```

Listing 3.1: Hill-type muscle model implementation.

3.1.3 The Force-Velocity Relationship

The second critical property of the contractile element is the force-velocity relationship, first characterized by Hill (1938) [13].

Biomechanical Principle 3.1. The Force-Velocity Asymmetry

The force-velocity relationship characterizes how muscle force depends on contraction velocity [13, 14]:

- **Concentric (shortening) contraction:** force *decreases* with increasing shortening velocity. At maximum shortening velocity v_0 , the muscle produces zero force. The relationship is well-approximated by a hyperbolic curve.
- **Eccentric (lengthening) contraction:** force *increases* beyond the maximum isometric force. The magnitude of enhancement is muscle and fiber-type dependent, documented in [23] and similar references.
- **Isometric ($v = 0$):** force equals the product of activation and the force-length relationship.

The asymmetry between concentric and eccentric force production is a fundamental property of biological muscle that is absent in engineered linear actuators.

3.1.4 Activation Dynamics

The neural command from the central nervous system does not instantly produce muscle force. The activation dynamics models the excitation-contraction coupling:

$$\dot{a}(t) = \frac{u(t) - a(t)}{\tau(u, a)}, \quad (3.2)$$

where $u(t) \in [0, 1]$ is the neural excitation, $a(t) \in [0, 1]$ is the activation level, and τ is a time constant that differs for activation and deactivation:

$$\tau(u, a) = \begin{cases} \tau_{\text{act}}(0.5 + 1.5a) & \text{if } u > a \text{ (activation),} \\ \tau_{\text{deact}}/(0.5 + 1.5a) & \text{if } u \leq a \text{ (deactivation).} \end{cases} \quad (3.3)$$

Typical values: $\tau_{\text{act}} = 10\text{--}15$ ms, $\tau_{\text{deact}} = 40\text{--}60$ ms [26]. The asymmetry between activation and deactivation time constants has important consequences for motor control: it is much easier to “turn on” a muscle quickly than to “turn it off.”

3.2 Tendon Mechanics

The tendon connects the muscle belly to bone. It is modeled as a nonlinear elastic element with a resting (slack) length ℓ_T^s :

$$F_T(\varepsilon_T) = \begin{cases} 0 & \varepsilon_T \leq 0, \\ F_0 \cdot \frac{e^{k_T \varepsilon_T} - 1}{e^{k_T \varepsilon_{0T}} - 1} & \varepsilon_T > 0, \end{cases} \quad (3.4)$$

where $\varepsilon_T = (\ell_T - \ell_T^s)/\ell_T^s$ is the tendon strain and $\varepsilon_{0T} \approx 0.033$ is the strain at maximum isometric force. The tendon stiffness parameter k_T is typically chosen so that the tendon strain is approximately 3.3% at F_0 [30].

Biomechanical Principle 3.2. Tendons as Energy Storage Elements

Biological tendons are not simply rigid force transmitters. They are viscoelastic elements that can store and return elastic strain energy. The energy storage capacity of a tendon is determined by its stiffness and the strain it undergoes during muscle contraction. This energy storage changes the energetics and the effective impedance of the muscle-tendon unit compared to a model with infinitely stiff tendons. The physiological consequence is that tendons enable more efficient movement by recovering elastic energy that would otherwise be dissipated, as documented in [2, 3].

3.3 Musculotendon Equilibrium

The muscle fiber and tendon must satisfy force equilibrium at all times. Assuming the muscle fiber acts at a pennation angle α to the tendon line of action:

$$F_T = (F_{\text{CE}} + F_{\text{PE}}) \cos \alpha, \quad (3.5)$$

where:

$$F_{CE} = a \cdot f_L(\tilde{\ell}^M) \cdot f_V(\tilde{v}^M) \cdot F_0, \quad (3.6)$$

$$F_{PE} = f_{PE}(\tilde{\ell}^M) \cdot F_0. \quad (3.7)$$

The musculotendon length ℓ^{MT} is the sum of tendon length and the projection of fiber length:

$$\ell^{MT} = \ell_T + \ell^M \cos \alpha. \quad (3.8)$$

Given a known musculotendon length (from the skeletal geometry) and activation, the equilibrium condition (3.5) and the constraint (3.8) determine the fiber length and velocity.

3.4 The Complete Hill Model as a Dynamical System

Combining all elements, the Hill muscle model can be expressed as a dynamical system with two state variables per muscle:

$$\dot{a} = \frac{u - a}{\tau(u, a)}, \quad (3.9)$$

$$\dot{\ell}^M = v^M(a, \ell^M, \ell^{MT}), \quad (3.10)$$

where v^M is obtained by inverting the force-velocity relationship at the equilibrium fiber force. The input is the neural excitation $u(t) \in [0, 1]$ and the “disturbance” is the musculotendon path length $\ell^{MT}(\mathbf{q})$, which depends on the joint configuration.

Biomechanical Principle 3.3. Muscle as a Nonlinear Control System

Each muscle is itself a nonlinear dynamical system with:

- **State:** $(a, \ell^M) \in [0, 1] \times \mathbb{R}_{>0}$
- **Input:** neural excitation $u \in [0, 1]$
- **Output:** tendon force F_T
- **Disturbance:** musculotendon path length $\ell^{MT}(\mathbf{q})$

The muscle transforms a scalar neural command into a scalar force, but the transformation is highly nonlinear, state-dependent, and history-dependent. The entire musculoskeletal system is a cascade: neural commands $\rightarrow$ muscle dynamics $\rightarrow$ tendon forces $\rightarrow$ joint torques $\rightarrow$ skeletal dynamics.

Exercises

1. **Force-Length Curve.** Plot the active force-length curve for $\gamma \in \{0.3, 0.45, 0.6\}$. For each, determine the “useful range” (the interval of fiber lengths where $f_L > 0.5$). How does the width parameter affect the operating range?

2. **Force-Velocity Curve.** Plot the complete force-velocity curve (both concentric and eccentric) for the biceps parameters given in the code listing. Mark the maximum eccentric force and the maximum shortening velocity.
3. **Activation Dynamics.** Simulate the activation dynamics for a square-wave neural excitation input ($u = 1$ for 200 ms, then $u = 0$). Plot the activation response with $\tau_{\text{act}} = 15$ ms and $\tau_{\text{deact}} = 50$ ms. How long does it take for activation to reach 95% of maximum? How long for deactivation?
4. **Tendon Compliance.** For the Achilles tendon (slack length $\ell_T^s = 0.25$ m, max force 4000 N), compute the tendon stretch at 50%, 75%, and 100% of maximum force. Does the tendon behave approximately linearly over this range?
5. **Isometric Force Surface.** Create a 3D surface plot of muscle force as a function of normalized fiber length ($\tilde{\ell}$) and activation (a) at zero velocity (isometric). Identify the point of maximum force production.
6. **(Programming.)** Implement a complete musculotendon simulation: given a prescribed musculotendon length trajectory $\ell^{MT}(t)$ and neural excitation $u(t)$, integrate the state equations (3.9) and (3.10) and plot the resulting tendon force $F_T(t)$.
7. **(Programming.)** Implement a two-muscle antagonist pair (e.g., biceps and triceps) acting on a single-DOF elbow joint. Given co-contraction ($u_{\text{biceps}} = u_{\text{triceps}} = 0.5$), compute the net joint torque and the joint stiffness. Verify that co-contraction increases stiffness without changing the equilibrium position.
8. **(Research.)** Compare the Thelen (2003) muscle model with the Millard et al. (2013) model used in OpenSim 4.x. What are the key differences in the force-velocity and passive force models? Which is more physiologically accurate?

Chapter 4

Joint Kinematics and Soft Tissue

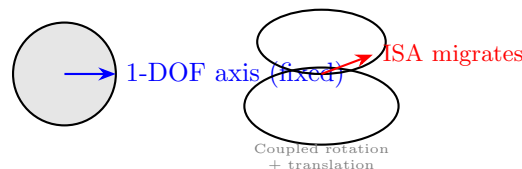
Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The knee is not a hinge. The shoulder is not a ball-and-socket. These are useful fictions that biology tolerates but does not enforce.

4.1 Beyond Ideal Joints

Engineering joints are designed to constrain motion to specific degrees of freedom: a revolute joint permits one rotation, a prismatic joint permits one translation. Biological joints achieve constraint through a fundamentally different mechanism: the interaction of articular surfaces, ligaments, joint capsule, and muscle forces. This section develops the kinematics of biological joints using the screw theory framework of Volume 0, extended to handle the additional complexity.

Engineering Revolute Biological Joint



4.1.1 The Instantaneous Screw Axis

Every rigid-body motion can be decomposed into a rotation about and translation along a **screw axis** (Chasles' theorem, Volume 0, Chapter 5). For an engineering revolute joint, this screw axis is fixed in both bodies. For a biological joint, the instantaneous screw axis (ISA) *moves* as the joint configuration changes.

Definition 4.1. Instantaneous Screw Axis of a Biological Joint

Given the relative twist $\mathcal{V}_{ij} = (\boldsymbol{\omega}, \mathbf{v}) \in \mathbb{R}^6$ between adjacent segments i and j , the instantaneous screw axis has:

- Direction: $\hat{\mathbf{s}} = \boldsymbol{\omega} / \|\boldsymbol{\omega}\|$
- Pitch: $h = \boldsymbol{\omega} \cdot \mathbf{v} / \|\boldsymbol{\omega}\|^2$
- Location: $\mathbf{q} = \boldsymbol{\omega} \times \mathbf{v} / \|\boldsymbol{\omega}\|^2$

For a pure rotation ($h = 0$), the ISA passes through $\mathbf{q}$ in direction $\hat{\mathbf{s}}$. Tracking how $\mathbf{q}$ and $\hat{\mathbf{s}}$ change with joint angle reveals the coupled kinematics of the biological joint.

4.1.2 The Knee: Rolling, Sliding, and Coupled Rotation

The tibiofemoral joint exemplifies the complexity of biological kinematics. During flexion from 0° to 90° :

1. The femoral condyles **roll** on the tibial plateaus (like a wheel on a road)
2. They simultaneously **slide** posteriorly (the contact point moves backward)
3. The tibia undergoes coupled **internal rotation** of approximately 15° (the “screw-home” mechanism)
4. The contact area shifts, changing the effective moment arm

This coupled motion means the knee ISA migrates superiorly and posteriorly during flexion. The position and velocity of the ISA are key outputs of kinematic analysis [6, 27]. The four-bar linkage model provides a simplified but insightful mechanical analog for understanding coupled knee kinematics [20]:

```

1 import numpy as np
2 from typing import Tuple
3
4 def compute_isa(
5     omega: np.ndarray,
6     v: np.ndarray
7 ) -> Tuple[np.ndarray, np.ndarray, float]:
8     """Compute the instantaneous screw axis from a twist.
9
10    Parameters
11    -----
12    omega : np.ndarray, shape (3,)
13           Angular velocity vector.
14    v : np.ndarray, shape (3,)
15       Linear velocity of the body-fixed frame origin.
16
17    Returns
18    -----
19    Tuple[np.ndarray, np.ndarray, float]
20        (point_on_axis, axis_direction, pitch)
21    """
22    omega_norm = np.linalg.norm(omega)
23    if omega_norm < 1e-10:

```

```

24     # Pure translation
25     direction = v / np.linalg.norm(v)
26     return np.zeros(3), direction, np.inf
27
28     direction = omega / omega_norm
29     pitch = np.dot(omega, v) / omega_norm**2
30     point = np.cross(omega, v) / omega_norm**2
31
32     return point, direction, pitch
33
34
35 def knee_isa_trajectory(
36     flexion_angles: np.ndarray,
37     roll_slide_ratio: float = 0.6
38 ) -> np.ndarray:
39     """Estimate knee ISA position during flexion.
40
41     Simple model: ISA migrates posteriorly and superiorly
42     with flexion, governed by the roll-to-slide ratio.
43
44     Parameters
45     -----
46     flexion_angles : np.ndarray
47         Array of flexion angles in degrees.
48     roll_slide_ratio : float
49         Ratio of rolling to total contact displacement.
50         Pure rolling = 1.0, pure sliding = 0.0.
51
52     Returns
53     -----
54     np.ndarray, shape (N, 3)
55         ISA positions in the tibial reference frame.
56     """
57     R_condyle = 0.030 # approximate femoral condyle radius (m)
58     isa_positions = np.zeros((len(flexion_angles), 3))
59
60     for i, theta in enumerate(np.radians(flexion_angles)):
61         # Posterior migration due to rolling
62         posterior = R_condyle * theta * roll_slide_ratio
63         # Superior migration (ISA moves up with flexion)
64         superior = R_condyle * (1.0 - np.cos(theta * 0.5))
65
66         isa_positions[i] = [-posterior, superior, 0.0]
67
68     return isa_positions

```

Listing 4.1: Instantaneous screw axis computation for a biological joint.

4.1.3 The Shoulder: The Most Complex Joint

The glenohumeral joint has the largest range of motion of any joint in the body, achieved at the expense of stability. The “ball-and-socket” description is a gross simplification: quantitative measurements show the humeral head translates during abduction [16, 9], and the instantaneous center of rotation shifts continuously. The instantaneous screw axis of the shoulder is not fixed and migrates through the joint during arm motion [11].

The shoulder complex involves four joints:

1. **Glenohumeral (GH)**: primary ball-and-socket, 3 DOF rotation + ~ 3 mm translation

2. **Acromioclavicular** (AC): 3 DOF rotation, small range
3. **Sternoclavicular** (SC): 3 DOF rotation, small range
4. **Scapulothoracic** (ST): not a true synovial joint; the scapula glides on the thorax, constrained as a *surface contact* rather than a pin joint

The scapulothoracic “joint” is particularly interesting from a mathematical perspective: it is a *holonomic constraint* that the scapula must remain on the thoracic surface, effectively acting as a 2-DOF constraint (the scapula can translate on the surface but cannot lift off).

4.2 Ligament Models

Ligaments constrain joint motion by resisting excessive displacement. They are modeled as nonlinear springs with:

$$F_{\text{lig}}(\varepsilon) = \begin{cases} 0 & \varepsilon \leq 0 \text{ (slack),} \\ k_1 \varepsilon^2 / (4\varepsilon_1) & 0 < \varepsilon \leq 2\varepsilon_1 \text{ (toe region),} \\ k_2(\varepsilon - \varepsilon_1) & \varepsilon > 2\varepsilon_1 \text{ (linear region),} \end{cases} \quad (4.1)$$

where $\varepsilon = (\ell - \ell_0)/\ell_0$ is the strain, ℓ_0 is the reference length, $\varepsilon_1 \approx 0.03$ is the transition strain, and k_1, k_2 are stiffness parameters [4].

4.3 Spinal Mechanics

The spine presents a unique modeling challenge: 24 vertebrae connected by intervertebral discs, each joint having 6 coupled degrees of freedom. The functional spinal unit (FSU) consisting of two adjacent vertebrae and their connecting disc is the basic mechanical unit.

The intervertebral disc consists of:

- **Nucleus pulposus**: a gel-like center that distributes compressive loads
- **Annulus fibrosus**: concentric rings of collagen fibers that resist torsion and bending

A common simplified model treats each FSU as a 6-DOF spring:

$$\mathbf{F}_{\text{disc}} = K_{\text{disc}} \cdot \boldsymbol{\delta}, \quad (4.2)$$

where $K_{\text{disc}} \in \mathbb{R}^{6 \times 6}$ is the stiffness matrix and $\boldsymbol{\delta} \in \mathbb{R}^6$ is the generalized displacement (3 translations + 3 rotations).

Exercises

1. **ISA Computation.** For a knee flexing at $60^\circ/\text{s}$ with 2 mm/s of posterior translation, compute the ISA direction, pitch, and location.
2. **Screw-Home Mechanism.** Plot the coupled internal rotation of the tibia as a function of knee flexion using the model: $\gamma = 15^\circ \cdot (1 - \cos(\theta/2))$ for $\theta \in [0, 90^\circ]$.

3. **Shoulder Kinematics.** Using screw axis theory, express the configuration of the hand (end-effector) as a product of exponentials through the SC, AC, GH, and elbow joints.
4. **(Programming.)** Implement the ligament force model and plot the force-strain curve. Identify the toe region and the linear region.
5. **(Programming.)** Create a 3D visualization of the knee ISA trajectory during flexion from 0° to 120° .

Chapter 5

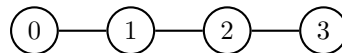
Multi-Body Dynamics of Biological Chains

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

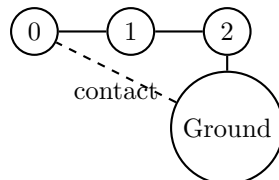
The human body is a high-dimensional articulated system with changing contacts and compliant tissues. Rigid-body dynamics is a useful approximation, but never the whole story.

5.1 Open-Chain vs. Closed-Chain Biomechanics

Most human movement involves open kinematic chains (the limbs) connected to a floating base (the pelvis/trunk). However, many functional activities create temporary closed chains: the stance phase of gait (foot on ground), pushing against a wall, or the golf swing when both hands grip the club.



Open Chain (arm reaching)



Closed Chain (gait stance phase)

5.1.1 Equations of Motion for Biological Chains

The equations of motion for an n -DOF musculoskeletal system take the standard Lagrangian form:

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + g(\mathbf{q}) = R(\mathbf{q})\mathbf{F}_{\text{muscle}} + J_c^T(\mathbf{q})\mathbf{F}_{\text{contact}}, \quad (5.1)$$

where:

- $M(\mathbf{q}) \in \mathbb{R}^{n \times n}$ is the mass matrix (from segment inertial properties)
- $C(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{n \times n}$ is the Coriolis/centrifugal matrix
- $g(\mathbf{q}) \in \mathbb{R}^n$ is the gravity vector
- $R(\mathbf{q}) \in \mathbb{R}^{n \times p}$ is the muscle moment arm matrix (p muscles, n DOF)
- $\mathbf{F}_{\text{muscle}} \in \mathbb{R}^p$ are the muscle forces (from Hill models, Chapter 3)
- $J_c(\mathbf{q})$ is the contact Jacobian
- $\mathbf{F}_{\text{contact}}$ are external contact forces

The key difference from Volume I's $\boldsymbol{\tau} = B\mathbf{u}$ is that joint torques are no longer direct control inputs. Instead:

$$\boldsymbol{\tau}_{\text{net}} = R(\mathbf{q})\mathbf{F}_{\text{muscle}} = \sum_{i=1}^p \mathbf{r}_i(\mathbf{q})F_{\text{muscle},i}, \quad (5.2)$$

where $\mathbf{r}_i(\mathbf{q}) \in \mathbb{R}^n$ is the i -th column of R , giving the moment arm vector of muscle i across all joints. This moment-arm mapping is the standard bridge between muscle models and generalized coordinates in musculoskeletal simulation [30, 8].

5.1.2 Bi-Articular Muscles

Unlike engineering actuators that span a single joint, many muscles cross two or more joints. These **bi-articular** (or multi-articular) muscles create mechanical coupling between joints that is not captured by the simple one-actuator-per-joint abstraction common in introductory robotics.

Biomechanical Principle 5.1. The Coordination Function of Bi-articular Muscles

Bi-articular muscles serve two distinct functions:

1. **Energy transfer:** they can transport mechanical energy from one joint to another without requiring a separate distal actuator. When the proximal joint extends (muscle shortening at the proximal end) and the distal joint flexes (muscle lengthening at the distal end), mechanical power can be redistributed from proximal to distal coordinates [30, 19]. The metabolic cost still depends on activation, contraction velocity, and force history.
2. **Coordination constraint:** they link joint motions, reducing the effective degrees of freedom and simplifying the control problem.

This is a form of *mechanical intelligence*—the morphology of the musculoskeletal system performs part of the control task. The gastrocnemius, for example, crosses both the knee and ankle, coupling knee extension to ankle plantarflexion during the push-off phase of gait.

Control-Theoretic Perspective: The matrix $R(\mathbf{q})$ can be read as a state-dependent input map from muscle forces to generalized torques. Bi-articular muscles may create dependencies among its columns, which is one reason the effective control directions in a musculoskeletal model can be lower-dimensional than the raw muscle count. Geometric control language is useful for describing that reduction [1], but the specific structure of $R(\mathbf{q})$ remains anatomy- and model-dependent.

```

1 import numpy as np
2
3 def compute_joint_torques(
4     moment_arms: np.ndarray,
5     muscle_forces: np.ndarray
6 ) -> np.ndarray:
7     """Compute net joint torques from muscle forces and moment arms.
8
9     Parameters
10    -----
11    moment_arms : np.ndarray, shape (n_joints, n_muscles)
12        Moment arm matrix  $R(\mathbf{q})$ . Each column gives the moment arm
13        of one muscle across all joints. Sign convention: positive
14        moment arm produces positive joint torque (typically flexion).
15    muscle_forces : np.ndarray, shape (n_muscles,)
16        Force produced by each muscle (always positive).
17
18    Returns
19    -----
20    np.ndarray, shape (n_joints,)
21        Net joint torques.
22    """
23    return moment_arms @ muscle_forces
24
25
26 # Example: 2-DOF elbow system with 4 muscles
27 # Joints: shoulder flexion, elbow flexion
28 # Muscles: anterior deltoid, biceps (biarticular),
29 #          triceps (biarticular), brachialis
30 #
31 # Convention: R has shape (n_joints, n_muscles) so that
32 # tau = R @ F maps the muscle-force vector to a joint-torque
33 # vector via a single matrix-vector product. Each row is a joint
34 # and each column is a muscle. This is the shape that
35 # compute_joint_torques() expects.
36 R = np.array([
37     # deltoid, biceps, triceps, brachialis
38     [0.05, 0.03, -0.025, 0.0], # row 0: shoulder moment arms (m)
39     [0.0, 0.04, -0.020, 0.03], # row 1: elbow moment arms (m)
40 ]) # Shape: (2 joints, 4 muscles) -- ready to use as-is
41
42 # Note: biceps has moment arms at BOTH joints (bi-articular)
43 muscle_forces = np.array([200.0, 150.0, 100.0, 120.0]) # Newtons
44
45 tau = compute_joint_torques(R, muscle_forces)

```

```

46 print(f"Shoulder torque: {tau[0]:.1f} N*m")
47 print(f"Elbow torque:      {tau[1]:.1f} N*m")

```

Listing 5.1: Computing joint torques from muscle forces through moment arms.

5.1.3 Co-Contraction and Joint Stiffness

When antagonist muscles co-contract (both agonist and antagonist are active simultaneously), the net joint torque may be zero but the joint **stiffness** increases. This is because each active muscle acts as a spring:

$$K_{\text{joint}} = \sum_{i=1}^p r_i^2(\mathbf{q}) \cdot \frac{\partial F_{\text{muscle},i}}{\partial \ell_i^M}, \quad (5.3)$$

where $\partial F / \partial \ell^M$ is the short-range stiffness of the muscle, which depends on activation level.

Co-contraction is metabolically expensive but biomechanically important: it allows the nervous system to modulate joint *impedance* independently of net joint *torque*. This is better understood as a biologically compliant analogue of impedance control rather than a literal duplicate of a rigid torque source [30, 18].

Position-dependent stiffness modulation gives a useful control vocabulary for discussing state-dependent actuation. Geometric control theory describes how state-dependent inputs change closed-loop structure [1], but the actual stiffness law in biomechanics still has to come from muscle, tendon, and contact modeling rather than from control theory alone.

5.2 The Mass Matrix of the Human Body

Computing the mass matrix $M(\mathbf{q})$ for a full musculoskeletal model follows the same recursive algorithms as Volume 0, Chapter 10 (the Composite Rigid Body Algorithm), using the segment inertial properties from Chapter 2. The key computational steps are:

1. Define the kinematic tree (segment connectivity and joint types)
2. Compute forward kinematics using the product of exponentials
3. Apply the CRBA to compute $M(\mathbf{q})$

For a 20-DOF model (pelvis + 2 legs + trunk + 2 arms), $M(\mathbf{q})$ is a 20×20 symmetric positive-definite matrix.

Exercises

1. **Bi-articular Energy Transfer.** For the gastrocnemius (crossing knee and ankle), show mathematically how knee extension can produce ankle torque through the bi-articular coupling.
2. **Co-Contraction Analysis.** Compute the joint stiffness when biceps and triceps co-contract at 50% activation with the elbow at 90° .

3. **Mass Matrix Properties.** For a 2-segment arm model, compute $M(\mathbf{q})$ at three configurations and verify positive-definiteness. Plot the condition number as a function of elbow angle.
4. **(Programming.)** Implement the CRBA for a 3-segment leg model using the de Leva segment properties. Compare with Pinocchio output.
5. **(Programming.)** Simulate a 2-DOF arm with 4 muscles (as in the code listing) performing a reaching movement. Use prescribed muscle activations and compute the resulting trajectory via forward dynamics.

Chapter 6

Inverse Problems in Biomechanics

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

In biomechanics, we almost never measure forces directly. We measure motion and infer the forces that caused it.

6.1 The Inverse Dynamics Pipeline

Inverse dynamics is one of the central tools in biomechanics [25, 8]: given measured kinematics $\mathbf{q}(t)$, $\dot{\mathbf{q}}(t)$, $\ddot{\mathbf{q}}(t)$ and external forces $\mathbf{F}_{\text{ext}}(t)$, compute the net joint torques. By itself, however, the calculation stops at net generalized loads; moving from those loads to individual muscle forces requires an additional redundancy-resolution or forward-simulation model. The complete pipeline, illustrated in Figure 6.1, transforms raw experimental data into progressively more model-dependent estimates:

$$\boldsymbol{\tau}(t) = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) - \mathbf{J}_c^T(\mathbf{q})\mathbf{F}_{\text{ext}}. \quad (6.1)$$

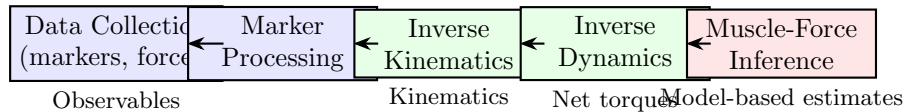


Figure 6.1: The Inverse Dynamics Pipeline: From Raw Experimental Data to Net Joint Moments and Subsequent Model-Based Muscle-Force Estimation.

The standard pipeline comprises five stages:

1. **Data Collection:** Motion capture markers + force plates
2. **Marker Processing:** Filtering, gap-filling, labeling

3. **Inverse Kinematics:** Fit skeleton model to marker data
4. **Inverse Dynamics:** Compute joint torques from RNEA
5. **Muscle Force Estimation:** Infer candidate muscle-force distributions using optimization or controller assumptions

Stages 1–4 are the standard inverse-kinematics/inverse-dynamics pipeline. Stage 5 is an additional modeling layer: it does not follow uniquely from the measured motion and external-force data, and different optimization or control assumptions can produce different muscle-force histories for the same net joint moments.

6.1.1 Inverse Kinematics

Given marker positions $\mathbf{m}_i^{\text{meas}}(t) \in \mathbb{R}^3$ for $i = 1, \dots, N_m$ markers, inverse kinematics solves:

$$\mathbf{q}^*(t) = \arg \min_{\mathbf{q}} \sum_{i=1}^{N_m} w_i \|\mathbf{m}_i^{\text{model}}(\mathbf{q}) - \mathbf{m}_i^{\text{meas}}(t)\|^2, \quad (6.2)$$

where $\mathbf{m}_i^{\text{model}}(\mathbf{q})$ is the predicted marker position from the skeleton model and w_i are weighting factors.

```

1 import numpy as np
2 from scipy.optimize import minimize
3
4 def inverse_kinematics_frame(
5     marker_positions_measured: np.ndarray,
6     marker_positions_model_func,
7     q_initial: np.ndarray,
8     weights: np.ndarray | None = None
9 ) -> np.ndarray:
10     """Solve inverse kinematics for a single frame.
11
12     Parameters
13     -----
14     marker_positions_measured : np.ndarray, shape (n_markers, 3)
15         Measured marker positions.
16     marker_positions_model_func : callable
17         Function q -> (n_markers, 3) predicted marker positions.
18     q_initial : np.ndarray, shape (n_dof,)
19         Initial guess for joint angles.
20     weights : np.ndarray, shape (n_markers,), optional
21         Per-marker weights.
22
23     Returns
24     -----
25     np.ndarray, shape (n_dof,)
26         Optimal joint angles.
27
28     """
29     n_markers = marker_positions_measured.shape[0]
30     if weights is None:
31         weights = np.ones(n_markers)
32
33     def objective(q: np.ndarray) -> float:
34         m_model = marker_positions_model_func(q)
35         residuals = m_model - marker_positions_measured
36         return float(np.sum(weights[:, None] * residuals**2))

```

```

36 result = minimize(objective, q_initial, method='L-BFGS-B')
37
38 return result.x

```

Listing 6.1: Simplified inverse kinematics solver.

6.1.2 Bottom-Up vs. Top-Down Inverse Dynamics

Bottom-up: start from the ground contact (where forces are measured by force plates) and propagate upward through the kinematic chain. Most accurate when force plate data is available.

Top-down: start from the most distal segment and work inward. Useful when ground reaction forces are not available (e.g., swimming, throwing).

6.2 The Muscle Redundancy Problem

Inverse dynamics gives net joint torques $\boldsymbol{\tau} \in \mathbb{R}^n$, but we want individual muscle forces $\mathbf{F} \in \mathbb{R}^p$ with $p > n$. The exact ratio between muscles and generalized coordinates depends on model granularity and how muscle groups are represented, but in all such cases the system is underdetermined:

$$R(\mathbf{q})\mathbf{F} = \boldsymbol{\tau}, \quad \mathbf{F} \geq 0. \quad (6.3)$$

From a modeling perspective, this is a constrained input-allocation problem. At a fixed configuration, the reachable torque set is the image of $R(\mathbf{q})$ intersected with the nonnegative muscle-force constraints. When $p > n$, multiple muscle-force vectors can produce the same joint torque history, so additional physiological or optimization assumptions are needed to select among them.

6.2.1 Static Optimization

The most common approach minimizes a cost function:

$$\min_{\mathbf{F}} \sum_{i=1}^p \left(\frac{F_i}{F_{0,i}} \right)^\alpha \quad \text{subject to} \quad R(\mathbf{q})\mathbf{F} = \boldsymbol{\tau}, \quad 0 \leq F_i \leq a_i \cdot f_L(\tilde{\ell}_i) \cdot F_{0,i}, \quad (6.4)$$

where $\alpha = 2$ (minimize sum of squared activations) or $\alpha = 3$ (minimize metabolic energy proxy). The constraint $F_i \leq a_i \cdot f_L \cdot F_{0,i}$ enforces the force-length and activation constraints from Chapter 3.

This optimization is one way to regularize the nonuniqueness of (6.3). The objective does not arise uniquely from the equations of motion; it encodes modeling priorities such as effort minimization, smoothness, or physiological plausibility. Different choices of α , reserve actuators, activation dynamics, or regularization can yield different force-sharing solutions while reproducing the same net torque history [30, 8].

6.2.2 Computed Muscle Control (CMC)

CMC [24] combines forward dynamics with feedback control. It uses a PD controller to track the measured kinematics, with the controller output being the desired muscle excitations:

$$\ddot{\mathbf{q}}_{\text{des}} = \ddot{\mathbf{q}}_{\text{meas}} + K_v(\dot{\mathbf{q}}_{\text{meas}} - \dot{\mathbf{q}}) + K_p(\mathbf{q}_{\text{meas}} - \mathbf{q}), \quad (6.5)$$

where K_v and K_p are velocity and position feedback gains.

CMC is a tracking algorithm rather than a direct measurement technique. It can produce dynamically consistent motion in a forward simulation, but its output depends on model fidelity, actuator limits, controller tuning, and how closely the measured kinematics can actually be realized by the muscle-driven model. General nonlinear-control results explain why feedback can reduce tracking error, yet they do not by themselves validate a particular muscle-force reconstruction [1, 24].

6.3 Parameter Estimation

Many biomechanical parameters (segment masses, muscle strengths, joint axis locations) are not directly measurable and must be estimated from motion data.

6.3.1 Bayesian Framework

The parameter estimation problem is naturally Bayesian:

$$p(\boldsymbol{\theta}|\mathbf{y}) \propto p(\mathbf{y}|\boldsymbol{\theta}) \cdot p(\boldsymbol{\theta}), \quad (6.6)$$

where $\boldsymbol{\theta}$ are model parameters, $\mathbf{y}$ are observations, $p(\mathbf{y}|\boldsymbol{\theta})$ is the likelihood, and $p(\boldsymbol{\theta})$ is the prior (from literature values and physical constraints).

Exercises

1. **Inverse Dynamics.** For a 2-segment pendulum model of the arm, compute the shoulder and elbow torques required during a reaching movement given $\mathbf{q}(t) = (\sin(\pi t/T), \pi t/(2T))$.
2. **Muscle Redundancy.** For a single-DOF elbow with 3 muscles (biceps, brachialis, brachioradialis), solve the static optimization for a given elbow torque. Show that the solution depends on the cost function exponent α .
3. **(Programming.)** Implement the static optimization for a 2-DOF, 6-muscle system. Compare $\alpha = 2$ and $\alpha = 3$ solutions.
4. **(Programming.)** Implement an inverse kinematics solver for a 3-segment arm model with 7 markers. Test with synthetic marker data.

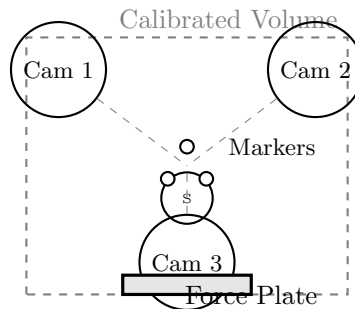
Chapter 7

Experimental Methods

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The quality of a biomechanical analysis is limited by the quality of the measurements, not the sophistication of the model.

7.1 Motion Capture Systems



7.1.1 Marker-Based Systems

Optoelectronic motion capture systems (Vicon, Qualisys, OptiTrack) track retroreflective markers attached to the body surface. Standard configurations use 30–80 markers placed on bony landmarks following established protocols (Plug-in-Gait, Cleveland Clinic, ISB).

Key specifications for biomechanical applications [5]:

- **Sampling rate:** 100–500 Hz for human motion (higher for impacts)
- **Spatial accuracy:** < 1 mm in calibrated volume
- **Marker size:** 9–14 mm diameter
- **Latency:** important for real-time applications (< 10 ms achievable)

It is important to note that the instrument accuracy above refers to the camera system alone. The dominant error in marker-based biomechanics is not camera noise but *soft tissue artifact*—the relative motion between skin markers and the underlying bone—discussed in Section 7.1.3 below [17].

7.1.2 Markerless Systems

Computer vision approaches (OpenPose, MediaPipe, DeepLabCut) estimate joint positions from video without physical markers. Accuracy is currently 10–30 mm, insufficient for clinical-grade inverse dynamics but useful for field measurements and large-scale studies.

```

1 import numpy as np
2 from scipy.signal import butter, filtfilt
3
4 def process_mocap_data(
5     marker_data: np.ndarray,
6     sample_rate: float,
7     cutoff_freq: float = 6.0,
8     filter_order: int = 4
9 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
10     """Process raw motion capture marker data.
11
12     Parameters
13     -----
14     marker_data : np.ndarray, shape (n_frames, n_markers, 3)
15         Raw marker positions in meters.
16     sample_rate : float
17         Sampling frequency in Hz.
18     cutoff_freq : float
19         Low-pass filter cutoff frequency in Hz.
20     filter_order : int
21         Butterworth filter order.
22
23     Returns
24     -----
25     tuple of np.ndarray
26         (positions, velocities, accelerations)
27         Each shape (n_frames, n_markers, 3).
28     """
29     # Design low-pass Butterworth filter
30     nyquist = sample_rate / 2.0
31     b, a = butter(filter_order, cutoff_freq / nyquist, btype='low')
32
33     n_frames, n_markers, _ = marker_data.shape
34     positions = np.zeros_like(marker_data)
35     velocities = np.zeros_like(marker_data)
36     accelerations = np.zeros_like(marker_data)
37
38     dt = 1.0 / sample_rate
39
40     for m in range(n_markers):
41         for axis in range(3):
42             # Filter position data
43             positions[:, m, axis] = filtfilt(
44                 b, a, marker_data[:, m, axis]
45             )
46
47     # Central difference for velocities
48     velocities[1:-1] = (positions[2:] - positions[:-2]) / (2 * dt)

```

```

49 velocities[0] = velocities[1]
50 velocities[-1] = velocities[-2]
51
52 # Central difference for accelerations
53 accelerations[1:-1] = (
54     positions[2:] - 2 * positions[1:-1] + positions[:-2]
55 ) / dt**2
56 accelerations[0] = accelerations[1]
57 accelerations[-1] = accelerations[-2]
58
59 return positions, velocities, accelerations

```

Listing 7.1: Motion capture data processing pipeline.

7.1.3 Soft Tissue Artifact

Soft tissue artifact (STA) is the relative motion between skin-mounted markers and the underlying bone caused by deformation of skin, subcutaneous fat, and muscle during movement. STA is widely recognized as the dominant source of error in marker-based motion capture, frequently exceeding the nominal camera resolution by more than an order of magnitude [5, 21].

The magnitude of STA varies strongly with marker location, anatomical region, task, and subject anthropometry. Representative values for the lower limb are 10–30 mm of skin-marker displacement relative to bone during walking and running, with the thigh segment the worst offender and the shank the best [5, 21]. STA is not random noise: it is task-correlated and heavily dependent on muscle contraction state, which means low-pass filtering cannot remove it.

Practical mitigation strategies include:

- **Rigid marker clusters** mounted on lightweight shells, which average out local skin motion over a neighborhood of markers.
- **Optimal pose estimation**, in which the rigid body pose of a segment is obtained by least-squares fitting of a cluster of markers rather than by differencing pairs of landmarks [5].
- **Global optimization / inverse kinematics** with joint constraints, which enforces an articulated model and redistributes STA across the chain.
- **Biplanar fluoroscopy** or bone pins when the research question demands sub-millimeter bone-to-bone accuracy.

STA should always be reported as part of the uncertainty budget of any kinetic analysis; inverse dynamics is sensitive to segment orientation errors, and moderate STA can produce large errors in computed joint moments.

7.1.4 Low-Pass Filter Cutoff Selection

Choosing the cutoff frequency for the low-pass Butterworth filter in `process_mocap_data()` is not a matter of taste: a cutoff that is too low attenuates the true signal (especially accelerations, whose energy extends to higher frequencies than positions), while a cutoff that is too high leaves noise that is amplified by numerical differentiation.

The standard objective procedure is **residual analysis**, due to Winter [25]. For a range of candidate cutoffs f_c , one computes the root-mean-square residual

$$R(f_c) = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i(f_c))^2},$$

where x_i is the raw signal and $\hat{x}_i(f_c)$ is the filtered signal. Plotting $R(f_c)$ vs. f_c produces a curve that is approximately linear in the high- f_c regime (residuals reflect only noise) and rises sharply at low f_c (residuals include signal). The optimal cutoff is the intersection of the extrapolated linear noise-only segment with the R -axis, typically in the range 4–10 Hz for kinematics of whole-body locomotion and 10–20 Hz for running and impact-dominated tasks [25]. Higher cutoffs are required when downstream analysis needs accelerations or joint powers, since differentiation acts as a high-pass amplifier.

7.2 Electromyography (EMG)

EMG measures the electrical activity of muscles, providing a window into neural commands. Surface EMG (sEMG) is non-invasive; intramuscular EMG (using fine-wire or needle electrodes) targets deep muscles.

7.2.1 EMG Processing

The raw EMG signal must be processed before use:

1. **Band-pass filter:** 20–450 Hz (remove movement artifact and noise)
2. **Full-wave rectification:** $|e(t)|$
3. **Envelope extraction:** low-pass filter at 3–6 Hz
4. **Normalization:** divide by maximum voluntary contraction (MVC) value

7.3 Force Plates

Force plates measure ground reaction forces (GRF) and moments. Standard specifications:

- 6-component measurement: $F_x, F_y, F_z, M_x, M_y, M_z$
- Sampling rate: 1000–2000 Hz
- Range: 0–5000 N vertical, 0–2500 N horizontal
- Center of pressure (CoP) accuracy: < 2 mm

The center of pressure is computed from force plate outputs:

$$x_{\text{CoP}} = -\frac{M_y + F_x \cdot d_z}{F_z}, \quad y_{\text{CoP}} = \frac{M_x - F_y \cdot d_z}{F_z}, \quad (7.1)$$

where d_z is the distance from the force plate surface to the transducer plane.

7.4 Inertial Measurement Units (IMUs)

IMUs combine accelerometers, gyroscopes, and magnetometers in a compact package. They provide:

- Angular velocity: $\boldsymbol{\omega}(t) \in \mathbb{R}^3$
- Linear acceleration: $\boldsymbol{a}(t) \in \mathbb{R}^3$
- Magnetic field: $\boldsymbol{B}(t) \in \mathbb{R}^3$ (for heading)

Orientation estimation from IMU data is a state estimation problem solvable using the extended Kalman filter (Volume I, Chapter 6) or complementary filters.

7.5 Data Synchronization

Multiple measurement systems must be temporally synchronized. Best practices:

- Hardware sync: trigger all systems from a common clock pulse
- Minimum: force plate and motion capture on the same clock
- EMG-to-motion delay: typically 10–80 ms (electromechanical delay)
- Cross-correlation for post-hoc alignment when hardware sync is unavailable

Exercises

1. **Filter Selection.** Using the provided code, filter a synthetic marker trajectory with cutoff frequencies of 4, 6, 8, and 12 Hz. Plot the effect on position, velocity, and acceleration. Discuss the trade-off between noise reduction and signal distortion.
2. **CoP Computation.** Given force plate data from a vertical jump (F_z peak = 3000 N), compute the center of pressure trajectory during takeoff.
3. **EMG Processing.** Process a synthetic EMG signal (sum of sinusoids + white noise) through the full processing pipeline. Plot each stage.
4. **(Programming.)** Implement an IMU orientation estimator using a complementary filter. Test with synthetic gyroscope and accelerometer data.

Chapter 8

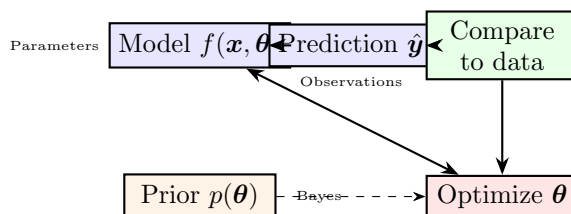
Inference on Biological Systems

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

We cannot open the body and read the parameters. We must infer them from the traces left by motion.

8.1 Parameter Identification From Motion Data

Biological system parameters (muscle strengths, tendon slack lengths, segment inertias) cannot typically be measured directly in vivo. They must be inferred from observable motion data using optimization and estimation techniques.



8.1.1 Maximum Likelihood Estimation

Given a parameterized model $\dot{\mathbf{x}} = f(\mathbf{x}, \boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$ and noisy observations $\mathbf{y}_k = h(\mathbf{x}_k) + \mathbf{v}_k$ at times t_k , the maximum likelihood estimate is:

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \sum_{k=1}^N \|\mathbf{y}_k - h(\mathbf{x}_k(\boldsymbol{\theta}))\|_{R^{-1}}^2, \quad (8.1)$$

where R is the measurement noise covariance.

8.1.2 Bayesian Estimation With Physical Constraints

The Bayesian framework (introduced in Chapter 6) is particularly valuable for biomechanical parameter estimation because it naturally incorporates prior knowledge from the literature:

```

1 import numpy as np
2 from scipy.optimize import minimize
3
4 def log_posterior(
5     theta: np.ndarray,
6     observations: np.ndarray,
7     forward_model,
8     prior_mean: np.ndarray,
9     prior_cov: np.ndarray,
10    noise_cov: np.ndarray
11 ) -> float:
12     """Compute the log-posterior for parameter estimation.
13
14     Parameters
15     -----
16     theta : np.ndarray
17         Model parameters.
18     observations : np.ndarray, shape (n_obs, n_dims)
19         Observed data.
20     forward_model : callable
21         theta -> predicted_observations.
22     prior_mean : np.ndarray
23         Prior parameter mean.
24     prior_cov : np.ndarray
25         Prior parameter covariance.
26     noise_cov : np.ndarray
27         Observation noise covariance.
28
29     Returns
30     -----
31     float
32         Negative log-posterior (for minimization).
33     """
34     # Log-likelihood
35     predicted = forward_model(theta)
36     residuals = observations - predicted
37     R_inv = np.linalg.inv(noise_cov)
38     log_likelihood = -0.5 * np.sum(residuals @ R_inv * residuals)
39
40     # Log-prior
41     prior_residuals = theta - prior_mean
42     P_inv = np.linalg.inv(prior_cov)
43     log_prior = -0.5 * prior_residuals @ P_inv @ prior_residuals
44
45     return -(log_likelihood + log_prior)
46
47
48 def map_estimate(
49     observations: np.ndarray,
50     forward_model,
51     prior_mean: np.ndarray,
52     prior_cov: np.ndarray,
53     noise_cov: np.ndarray,
54     bounds: list[tuple[float, float]] | None = None
55 ) -> np.ndarray:
56     """Compute Maximum A Posteriori (MAP) parameter estimate.

```

```

57
58     Parameters
59     -----
60     observations : np.ndarray
61         Observed data.
62     forward_model : callable
63         Parameter-to-observation mapping.
64     prior_mean : np.ndarray
65         Prior mean from literature.
66     prior_cov : np.ndarray
67         Prior uncertainty.
68     noise_cov : np.ndarray
69         Measurement noise covariance.
70     bounds : list of tuple, optional
71         Physical bounds on parameters.
72
73     Returns
74     -----
75     np.ndarray
76         MAP parameter estimate.
77     """
78     result = minimize(
79         log_posterior,
80         prior_mean,
81         args=(observations, forward_model, prior_mean,
82             prior_cov, noise_cov),
83         method='L-BFGS-B',
84         bounds=bounds
85     )
86     return result.x

```

Listing 8.1: Bayesian parameter estimation for musculoskeletal model.

8.2 System Identification for Biological Systems

System identification techniques from control theory (Volume I) apply to biomechanical systems with modifications:

1. **Joint impedance identification:** Perturb the limb and measure the response. The stiffness, damping, and inertia of the joint can be estimated from the transfer function.
2. **Muscle parameter identification:** Optimize muscle parameters (F_0 , optimal length, tendon slack length) to match experimental force or torque data.
3. **Neural control identification:** Estimate the feedback gains used by the nervous system from perturbation experiments.

8.3 Machine Learning Approaches

Modern ML approaches to biomechanical inference include:

- Physics-informed neural networks (PINNs) that enforce equations of motion
- Gaussian process regression for joint angle estimation from sparse data
- Deep learning for markerless motion capture pose estimation

Exercises

1. **MAP Estimation.** Using the provided code, estimate the optimal fiber length of a biceps muscle from simulated isometric force data at 5 joint angles.
2. **Identifiability Analysis.** For a Hill-type muscle model, analyze which parameters can be uniquely estimated from isometric force-angle data alone. Which require dynamic data?
3. **(Programming.)** Implement a joint impedance identification experiment: simulate a perturbed pendulum and estimate stiffness and damping from the response.

Chapter 9

Deformable Bodies and Continuum Mechanics

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

When the rigid-body assumption fails, we do not abandon mechanics. We embrace the continuum.

9.1 When Rigid-Body Assumptions Fail

The rigid-body model fails when:

1. **Internal stresses are needed:** to predict injury, fracture, or tissue damage
2. **Deformation affects kinematics:** shaft flexibility in golf, spine compliance
3. **Wave propagation matters:** impact biomechanics, blast injury
4. **Fluid-structure interaction:** blood flow, synovial fluid, respiratory mechanics

9.2 Beam Models for Long Bones and Shafts

When deformation is small and the body is slender (length $\gg$ diameter), beam theory provides an efficient model. A schematic representation of the finite element beam model is shown in Figure 9.1.

For a Euler-Bernoulli beam:

$$EI \frac{\partial^4 w}{\partial x^4} + \rho A \frac{\partial^2 w}{\partial t^2} = f(x, t), \quad (9.1)$$

where E is the elastic modulus, I is the second moment of area, ρ is density, A is cross-sectional area, $w(x, t)$ is transverse displacement, and f is the distributed load.

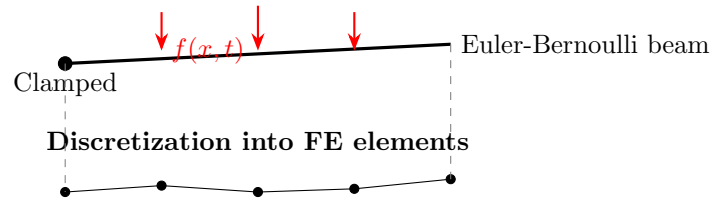


Figure 9.1: Finite Element Beam Model: Continuous Euler-Bernoulli Beam (Top) and Its Discretization Into Nodal Elements (Bottom).

9.2.1 Modal Analysis

The solution is expanded in modes:

$$w(x, t) = \sum_{k=1}^N \phi_k(x) \eta_k(t), \quad (9.2)$$

where $\phi_k(x)$ are mode shapes and $\eta_k(t)$ are modal coordinates. This converts the PDE to a system of ODEs that can be appended to the rigid-body equations of motion.

9.2.2 Golf Club Shaft Flexibility

The golf club shaft is the most accessible example of a flexible link in sport biomechanics. A typical shaft has:

- Length: ~ 1.1 m
- First bending mode: 4–6 Hz (driver), 6–10 Hz (irons)
- Tip deflection at impact: 5–15 cm
- Lead/lag deflection: 2–5 cm
- Toe-down droop: 3–8 cm (gravity effect during downswing)

```

1 import numpy as np
2 from scipy.linalg import eigh
3
4 def shaft_modal_analysis(
5     n_elements: int = 20,
6     length: float = 1.1,
7     EI_root: float = 50.0,
8     EI_tip: float = 15.0,
9     rho_A: float = 0.15
10 ) -> tuple[np.ndarray, np.ndarray]:
11     """Compute natural frequencies and mode shapes of a tapered shaft.
12
13     Uses finite element beam model with linear EI taper.
14
15     Parameters
16     -----
17     n_elements : int
18         Number of finite elements.

```

```

19     length : float
20         Shaft length in meters.
21     EI_root : float
22         Flexural rigidity at root (grip end) in N*m^2.
23     EI_tip : float
24         Flexural rigidity at tip (head end) in N*m^2.
25     rho_A : float
26         Mass per unit length in kg/m.
27
28     Returns
29     -----
30     tuple of np.ndarray
31         (frequencies_hz, mode_shapes)
32     """
33     n_nodes = n_elements + 1
34     n_dof = 2 * n_nodes # displacement + rotation at each node
35     h = length / n_elements
36
37     K = np.zeros((n_dof, n_dof))
38     M = np.zeros((n_dof, n_dof))
39
40     for e in range(n_elements):
41         # Local EI for this element (linear taper)
42         xi = (e + 0.5) / n_elements
43         EI_local = EI_root + (EI_tip - EI_root) * xi
44
45         # Euler-Bernoulli beam element stiffness matrix
46         k_e = EI_local / h**3 * np.array([
47             [12, 6*h, -12, 6*h],
48             [6*h, 4*h**2, -6*h, 2*h**2],
49             [-12, -6*h, 12, -6*h],
50             [6*h, 2*h**2, -6*h, 4*h**2]
51         ])
52
53         # Consistent mass matrix
54         m_e = rho_A * h / 420 * np.array([
55             [156, 22*h, 54, -13*h],
56             [22*h, 4*h**2, 13*h, -3*h**2],
57             [54, 13*h, 156, -22*h],
58             [-13*h, -3*h**2, -22*h, 4*h**2]
59         ])
60
61         # Assembly
62         dofs = [2*e, 2*e+1, 2*e+2, 2*e+3]
63         for i_idx, i in enumerate(dofs):
64             for j_idx, j in enumerate(dofs):
65                 K[i, j] += k_e[i_idx, j_idx]
66                 M[i, j] += m_e[i_idx, j_idx]
67
68         # Boundary conditions: clamped at root (grip)
69         free_dofs = list(range(2, n_dof)) # Remove first node
70         K_free = K[np.ix_(free_dofs, free_dofs)]
71         M_free = M[np.ix_(free_dofs, free_dofs)]
72
73         # Solve eigenvalue problem
74         eigenvalues, eigenvectors = eigh(K_free, M_free)
75
76         # Convert to Hz
77         frequencies = np.sqrt(np.abs(eigenvalues)) / (2 * np.pi)
78

```

```

79     return frequencies[:6], eigenvectors[:, :6]
80
81
82 # Example
83 freqs, modes = shaft_modal_analysis()
84 print("Natural frequencies (Hz):")
85 for i, f in enumerate(freqs):
86     print(f"  Mode {i+1}: {f:.1f} Hz")

```

Listing 9.1: Modal analysis of a golf club shaft.

9.3 Finite Element Methods (Overview)

For complex geometries (vertebral bodies, skull, articular surfaces), the finite element method (FEM) provides the most general approach. While a full treatment is beyond this text's scope, the key concept is:

The continuous domain Ω is divided into elements, and the displacement field is approximated using shape functions:

$$\mathbf{u}(\mathbf{x}, t) \approx \sum_i N_i(\mathbf{x}) \mathbf{u}_i(t), \quad (9.3)$$

leading to the assembled system:

$$M_{\text{FEM}} \ddot{\mathbf{u}} + C_{\text{FEM}} \dot{\mathbf{u}} + K_{\text{FEM}} \mathbf{u} = \mathbf{f}_{\text{ext}}. \quad (9.4)$$

Exercises

1. **Shaft Frequency.** Using the provided code, compute how the first natural frequency changes as the tip-to-root EI ratio varies from 0.1 to 0.9. Plot the result.
2. **Mode Shapes.** Plot the first three mode shapes of the shaft. Identify nodes (zero-crossings) and antinodes (maximum displacement).
3. **Timescale Separation.** If the golf downswing takes 0.25 s, estimate the number of oscillation cycles for each of the first 3 modes. Which modes are quasi-static?
4. **(Programming.)** Add a tip mass (clubhead, 0.2 kg) to the shaft FEM model and recompute the natural frequencies. How does the clubhead mass affect the frequencies?

Chapter 10

Applications of Control Theory to Biological Systems

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The body is not a robot that happens to be made of meat. It is a system that has evolved to exploit its own physics.

10.1 Forward Dynamics Simulation

Forward dynamics simulation integrates the equations of motion given muscle excitations $\mathbf{u}(t)$. A feedback control loop for biological systems is illustrated in Figure 10.1.

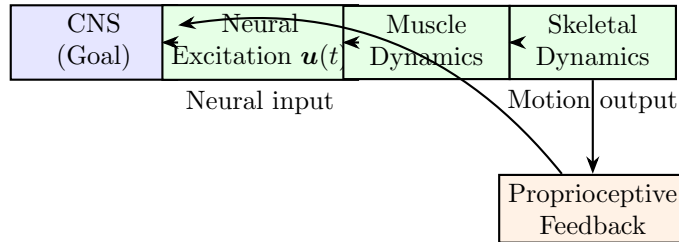


Figure 10.1: Biological Feedback Control Loop: CNS Goal Signals Drive Neural Excitations Through Muscle and Skeletal Dynamics, With Proprioceptive Feedback Closing the Loop.

$$\ddot{\mathbf{q}} = M(\mathbf{q})^{-1} [R(\mathbf{q})\mathbf{F}_{\text{muscle}} + J_c^T \mathbf{F}_c - C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - g(\mathbf{q})], \quad (10.1)$$

coupled with the muscle state equations (Chapter 3):

$$\dot{a}_i = (u_i - a_i)/\tau_i, \quad (10.2)$$

$$\dot{\ell}_i^M = v_i^M(a_i, \ell_i^M, \ell_i^{MT}(\mathbf{q})). \quad (10.3)$$

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 from typing import Callable
4
5 def forward_dynamics_simulation(
6     mass_matrix_func: Callable,
7     coriolis_func: Callable,
8     gravity_func: Callable,
9     moment_arm_func: Callable,
10    muscle_force_func: Callable,
11    excitation_func: Callable,
12    q0: np.ndarray,
13    qdot0: np.ndarray,
14    a0: np.ndarray,
15    t_span: tuple[float, float],
16    n_eval: int = 500
17 ) -> dict:
18     """Simulate forward dynamics of a musculoskeletal system.
19
20     Parameters
21     -----
22     mass_matrix_func : callable
23         q -> M(q), shape (n_dof, n_dof)
24     coriolis_func : callable
25         (q, qdot) -> C(q, qdot) @ qdot, shape (n_dof,)
26     gravity_func : callable
27         q -> g(q), shape (n_dof,)
28     moment_arm_func : callable
29         q -> R(q), shape (n_dof, n_muscles)
30     muscle_force_func : callable
31         (a, l_M, v_M) -> F_muscle for each muscle
32     excitation_func : callable
33         t -> u(t), shape (n_muscles,)
34     q0 : np.ndarray
35         Initial joint angles.
36     qdot0 : np.ndarray
37         Initial joint velocities.
38     a0 : np.ndarray
39         Initial muscle activations.
40     t_span : tuple
41         (t_start, t_end)
42     n_eval : int
43         Number of output time points.
44
45     Returns
46     -----
47     dict
48         Keys: 't', 'q', 'qdot', 'a', 'tau'
49     """
50     n_dof = len(q0)
51     n_muscles = len(a0)
52     t_eval = np.linspace(t_span[0], t_span[1], n_eval)
53
54     def dynamics(t: float, state: np.ndarray) -> np.ndarray:
55         q = state[:n_dof]
56         qdot = state[n_dof:2*n_dof]
57         a = np.clip(state[2*n_dof:], 0.0, 1.0)
58
59         # Muscle forces (simplified: assume known fiber length)
60         F_muscle = np.array([

```

```

61         a[i] * 500.0 # Simplified: F = a * F0
62         for i in range(n_muscles)
63     ])
64
65     # Joint torques
66     R = moment_arm_func(q)
67     tau = R @ F_muscle
68
69     # Skeletal dynamics
70     M = mass_matrix_func(q)
71     C_qdot = coriolis_func(q, qdot)
72     g = gravity_func(q)
73     qddot = np.linalg.solve(M, tau - C_qdot - g)
74
75     # Activation dynamics
76     u = excitation_func(t)
77     tau_act = 0.015 # Illustrative placeholder; replace with a cited
model value.
78     tau_deact = 0.050 # Illustrative placeholder; replace with a cited
model value.
79     adot = np.where(
80         u > a,
81         (u - a) / tau_act,
82         (u - a) / tau_deact
83     )
84
85     return np.concatenate([qdot, qddot, adot])
86
87     state0 = np.concatenate([q0, qdot0, a0])
88     sol = solve_ivp(dynamics, t_span, state0, t_eval=t_eval,
89                     method='RK45', max_step=0.001)
90
91     return {
92         't': sol.t,
93         'q': sol.y[:n_dof].T,
94         'qdot': sol.y[n_dof:2*n_dof].T,
95         'a': sol.y[2*n_dof:].T,
96     }

```

Listing 10.1: Forward dynamics integration for a musculoskeletal model.

10.2 Predictive Simulation

Predictive simulation uses trajectory optimization (Volume I, Chapter 5) to find the muscle excitations that produce a desired motion while minimizing a cost:

$$\min_{\mathbf{u}(\cdot)} \int_0^T \mathcal{L}(\mathbf{x}, \mathbf{u}) dt \quad \text{s.t.} \quad \dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}), \mathbf{x}(0) = \mathbf{x}_0, h(\mathbf{x}, \mathbf{u}) \leq 0, \quad (10.4)$$

where $\mathcal{L}$ is a physiologically motivated cost (e.g., metabolic energy, muscle fatigue, deviation from desired kinematics) and h captures physical constraints (joint limits, muscle force capacity, contact constraints).

Agrachev & Sachkov [1], Theorem 8.1 (Pontryagin's Maximum Principle), establishes the fundamental existence and optimality conditions for such optimal control problems. Their treatment covers affine systems with constraints and provides both theoretical guarantees and computational methods (the method of characteristics) for solving OCPs of this form.

10.3 Prosthetics and Exoskeleton Control

The control theory of Volumes I–II applies directly to assistive devices:

- **Powered prostheses:** trajectory tracking with impedance modulation
- **Exoskeletons:** assist-as-needed control using contraction theory
- **FES (Functional Electrical Stimulation):** direct muscle activation control for paralyzed limbs

10.4 Sport Biomechanics Applications

10.4.1 The Golf Swing as a Biomechanical System

The golf swing demonstrates every concept in this volume:

1. Multi-segment kinematic chain (15+ DOF)
2. Muscle-driven actuation with co-contraction
3. Bi-articular coupling (forearm muscles, lats)
4. Passive dynamics exploitation (wrist release)
5. Shaft flexibility (deformable body)
6. Ground reaction force coupling (closed chain at the feet)

The complete biomechanical analysis integrates:

- Motion capture (kinematics)
- Force plates (ground reaction forces)
- EMG (muscle activation patterns)
- Inverse dynamics (joint torques)
- Static optimization (muscle forces)
- Forward dynamics validation (predictive simulation)

10.5 Synthesis: From Theory to Application

The progression through this volume mirrors the progression through the series:

Volume	Concept	Biomechanical Application	Agrachev & Sachkov
Vol 0	Screw axes, PoE	Joint kinematics, ISA analysis	
Vol I	Contraction theory	Stability of muscle-driven systems	
Vol I	DDP/iLQR	Optimal muscle excitation patterns	
Vol II	Trajectory tubes	Motor variability envelopes	
Vol II	Phase variables	Gait cycle parameterization	
Vol III	Hill model	Force prediction	
Vol III	Inverse dynamics	Joint torque estimation	
Vol IV	Motor control	Neural command generation	

Exercises

1. **Forward Simulation.** Using the provided code (adapted for a simple 2-DOF model), simulate a reaching movement with prescribed muscle excitation ramps.
2. **Predictive vs. Tracking.** Compare the muscle activations from static optimization (tracking measured kinematics) with those from predictive simulation (optimizing a metabolic cost).
3. **Prosthetic Control.** Design an impedance controller for a prosthetic knee that provides different stiffness during stance and swing phases.
4. **(Programming.)** Implement a complete inverse dynamics $\rightarrow$ static optimization pipeline for a 3-DOF planar arm model with 6 muscles.

Bibliography

- [1] Andrei A. Agrachev and Yuri L. Sachkov. *Control Theory from the Geometric Viewpoint*, volume 87 of *Encyclopaedia of Mathematical Sciences*. Springer, 2004.
- [2] R. M. Alexander. Energy-saving mechanisms in walking and running. *Journal of Experimental Biology*, 160:55–69, 1991.
- [3] R. M. Alexander. Tendon elasticity and muscle function. *Comparative Biochemistry and Physiology. Part A, Molecular and Integrative Physiology*, 133(4):1001–1011, 2002.
- [4] L. Blankevoort and R. Huiskes. Ligament-bone interaction in a three-dimensional model of the knee. *Journal of Biomechanical Engineering*, 113(3):263–269, 1991.
- [5] Lorenzo Chiari, Ugo Della Croce, Alberto Leardini, and Aurelio Cappozzo. Human movement analysis using stereophotogrammetry. part 2: Instrumental errors. *Gait and Posture*, 21(2):197–211, 2005.
- [6] J. J. Crisco, J. C. Coburn, T. J. Wolfson, W. H. Akeson, and S. L.-Y. Woo. Instantaneous center of rotation as a function of a four-bar linkage knee model. *Journal of Biomechanics*, 34(6):735–742, 2000.
- [7] P. de Leva. Adjustments to zatsiorsky-seluyanov’s segment inertia parameters. *Journal of Biomechanics*, 29(9):1223–1230, 1996.
- [8] S. L. Delp, F. C. Anderson, A. S. Arnold, et al. Opensim: Open-source software to create and analyze dynamic simulations of movement. *IEEE Transactions on Biomedical Engineering*, 54(11):1940–1950, 2007.
- [9] C. R. Dickerson, D. B. Chaffin, and R. E. Hughes. Accurate calculation of scapular anatomy with 3-dimensional computed tomography. *Journal of Shoulder and Elbow Surgery*, 14(6):601–609, 2005.
- [10] E. S. Grood and W. J. Suntay. A joint coordinate system for the clinical description of three-dimensional motions: application to the knee. *Journal of Biomechanical Engineering*, 105(2):136–144, 1983.
- [11] A. M. Halder, E. Itoi, and K. N. An. Structural and functional anatomy of the shoulder. *The Shoulder*, 2000.
- [12] M. E. Harrington, A. B. Zavatsky, S. E. M. Lawson, Z. Yuan, and T. N. Theologis. Prediction of the hip joint centre in adults, children, and patients with cerebral palsy based on magnetic resonance imaging. *Journal of Biomechanics*, 40(3):595–602, 2007.
- [13] A. V. Hill. The heat of shortening and the dynamic constants of muscle. *Proceedings of the Royal Society B: Biological Sciences*, 126(843):136–195, 1938.
- [14] A. V. Hill. A note on the mechanics of the contraction of skeletal muscle. *Proceedings of the Royal Society B: Biological Sciences*, 137:273–280, 1950.

- [15] A. F. Huxley and R. M. Simmons. Proposed mechanism of force generation in striated muscle. *Nature*, 233:533–538, 1971.
- [16] A. R. Karduna, P. W. McClure, L. A. Michener, and B. Sennett. Scapular kinematics and humeral head translation in the throwing shoulder. *Clinical Biomechanics*, 15(10):789–798, 2000.
- [17] Alberto Leardini, Lorenzo Chiari, Ugo Della Croce, and Aurelio Cappozzo. Human movement analysis using stereophotogrammetry. part 3: Soft tissue artifact assessment and compensation. *Gait and Posture*, 21(2):212–225, 2005.
- [18] Matthew Millard, Thomas Uchida, Ajay Seth, and Scott L. Delp. Flexing computational muscle: Modeling and simulation of musculotendon dynamics. *Journal of Biomechanical Engineering*, 135(2):021005, 2013.
- [19] Richard R. Neptune, Steven A. Kautz, and Felix E. Zajac. Contributions of the individual ankle plantar flexors to support, forward progression and swing initiation during walking. *Journal of Biomechanics*, 34(11):1387–1398, 2001.
- [20] J. J. O'Connor, E. A. Shercliff, E. Biden, and J. P. Paul. Geometry of the knee in the sagittal plane. *Proceedings of the Institution of Mechanical Engineers. Part H*, 203(4):223–233, 1989.
- [21] Alana Peters, Brook Galna, Morgan Sangeux, Meg Morris, and Richard Baker. Quantification of soft tissue artifact in lower limb human motion analysis: A systematic review. *Gait and Posture*, 31(1):1–8, 2010.
- [22] J. P. Scholz and G. Schöner. The uncontrolled manifold concept: Identifying control variables for a functional task. *Experimental Brain Research*, 126:289–306, 1999.
- [23] D. G. Thelen. Adjustment of muscle mechanics model parameters to simulate dynamic contractions in older adults. *Journal of Biomechanical Engineering*, 125(1):70–77, 2003.
- [24] Darryl G. Thelen, Frank C. Anderson, and Scott L. Delp. Generating dynamic simulations of movement using computed muscle control. *Journal of Biomechanics*, 36(3):321–328, 2003.
- [25] David A. Winter. *Biomechanics and Motor Control of Human Movement*. John Wiley & Sons, Hoboken, NJ, 4th edition, 2009.
- [26] J. M. Winters and L. Stark. Different roles of fast and slow twitch muscle fibers in a level versus graded running. *Journal of Biomechanics*, 17(2):119–131, 1984.
- [27] P. Wretenberg, D. K. Ramsey, and G. Nemeth. Knee movement tracking using accelerometers and gyroscopes. *Medicine and Science in Sports and Exercise*, 27(10):1474–1479, 1995.
- [28] G. Wu, S. Siegler, P. Allard, et al. Isb recommendation on definitions of joint coordinate system of various joints for the reporting of human joint motion—part i: ankle, hip, and spine. *Journal of Biomechanics*, 35(4):543–548, 2002.

- [29] G. Wu, F. C. T. van der Helm, H. E. J. Veeger, et al. Isb recommendation on definitions of joint coordinate systems of various joints for the reporting of human joint motion — part ii: shoulder, elbow, wrist and hand. *Journal of Biomechanics*, 38(5):981–992, 2005.
- [30] F. E. Zajac. Muscle and tendon: properties, models, scaling, and application to biomechanics and motor control. *Critical Reviews in Biomedical Engineering*, 17(4):359–411, 1989.